Introduction to React.js

Learn the basics of React, one of the most popular JavaScript frameworks out there. It's used by companies large and small including Instagram, Netflix, Airbnb and many more!

WE'LL COVER THE FOLLOWING

- Why React.js?
- What we'll build

Why React.js?

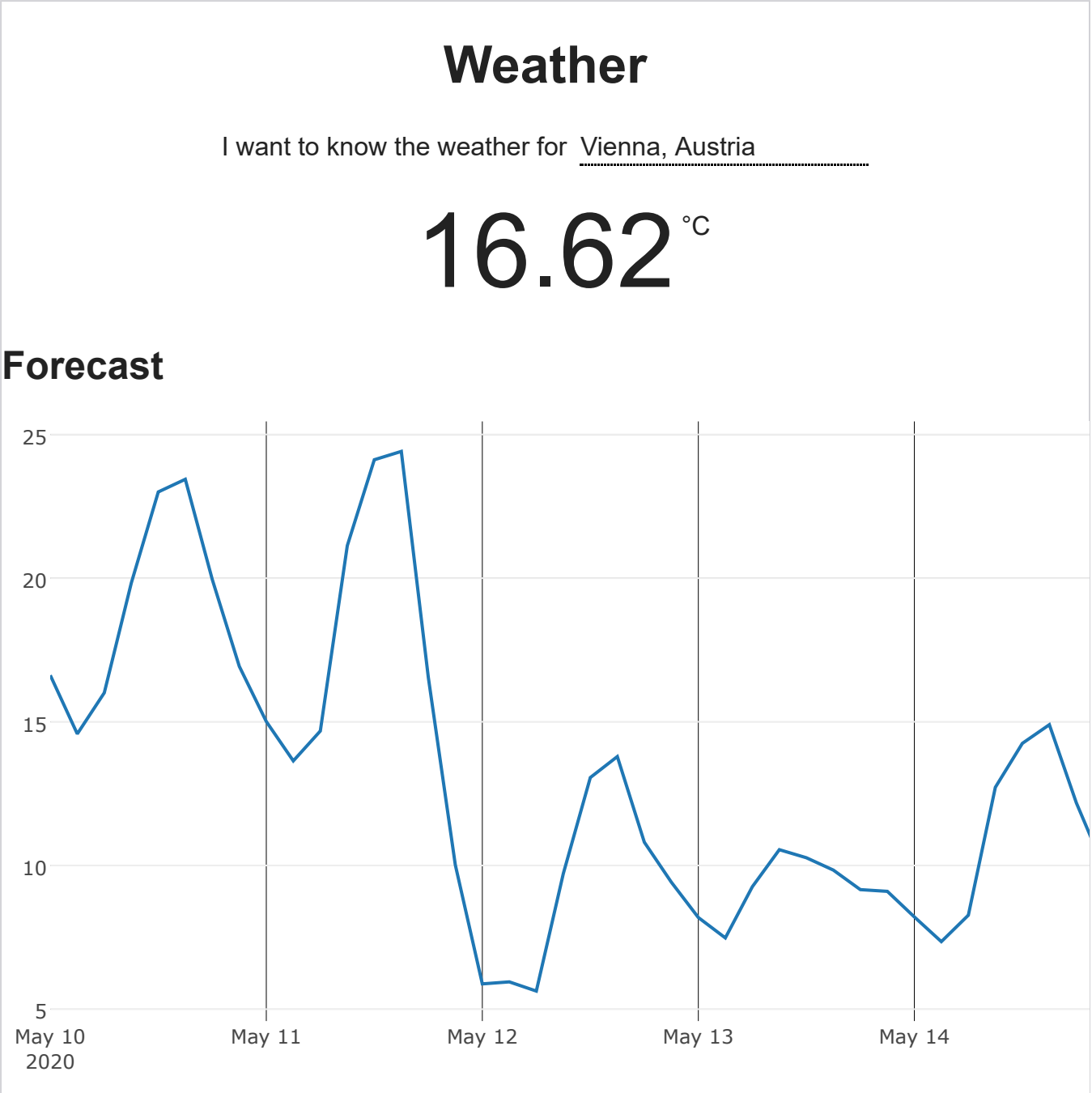
React, to quote Facebook themselves, was built to solve one problem: "building large applications with data that changes over time". (that's why Instagram and Facebook are built with it)

With React, it's simple to express how your app should look at any given point in time. It will figure out which parts need to update when the data changes for you, and will only render those.

It also popularised building apps with encapsulated, reusable and composable components. It's a different way of thinking and of going about building webapps, but once you're used to it you can never go back!

What we'll build

This is the app we'll be building in this tutorial series (yes, this is a live example! Click around and explore):



In this first chapter of the tutorial we'll go through the React basics, before starting to build the above app in chapter two. If you feel comfortable with React and JSX feel free to skip ahead but **I highly recommend going through this chapter to make sure you know React**!

Getting started with React

Let's render a Hello World div using React

WE'LL COVER THE FOLLOWING

- `ReactDOM.render()`
- `React.createElement()`

React consists of two libraries, `React` and `ReactDOM`. `React` allows you to create elements, which we render with `ReactDOM`. They are split because you could (theoretically) render those ReactElements anywhere, not only to the browser DOM.

> For example, there are initial experiments out there for rendering React to Canvas, WebVR and even hardware!

Take a look at the following HTML Page.

HTML CSS JavaScript

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>React</title>
  <link rel="stylesheet" href="//cdnjs.cloudflare.c
  <script src="//fb.me/react-with-addons-0.14.3.js"
  <script src="//fb.me/react-dom-0.14.3.js"></script
</head>
<body>
  <div id="container"></div>
</body>
</html>
```

```javascript
ReactDOM.render(
  React.createElement('h1', { className: 'hea
  document.getElementById('container')
);
```

output

Hello World!

You will see an `<h1>` with the text “Hello World”. This is the source code generating that text:

```
ReactDOM.render(
  React.createElement('h1', {className: 'heading'}, 'Hello World'),
  document.getElementById('container')
);
```

We use the `ReactDOM.render()` function to render a ReactElement created with the `React.createElement()` function.

`ReactDOM.render()`

The `ReactDOM.render()` function takes two arguments: The ReactElement to render, and the DOM node we want to render into. (the “entry point”)

```
ReactDOM.render(
  React.createElement('h1', {className: 'heading'}, 'Hello World'),
  document.getElementById('container')
);
```

Now you might think creating a `ReactDOM.render()` function for every ReactElement you have is the way to go. That’s not a very good idea – it empties the DOM node we use as an entry point. How do we render multiple ReactElements then? Let’s take a look at the `React.createElement()` function to figure it out!

`React.createElement()`

This function takes the node (or ReactElement) we want to create as the first argument and some properties (like `type`) in an object as the second

argument:

```
React.createElement('input');
// -> <input></input>
React.createElement('input', { type: 'radio' });
// -> <input type="radio"></input>
React.createElement('input', { className: 'heading', type: 'radio' });
// -> <input class="heading" type="radio"></input>
```

> Notice how the HTML `class` attribute has to be set via `className` property in react. This is because `class` is a reserved keyword in JavaScript, which means we might introduce unwanted problems into our apps by passing in `class`. React mitigates this by using the `className` property instead of `class`!

We can also (optionally) pass children as the third argument! The simplest usecase here is to render a bit of text:

```
React.createElement('h1', null, 'Hello World');
// -> <h1>Hello World</h1>
```

The children (above: `'Hello World'`) can also be another ReactElement! Let's say we want to add a `<div class="wrapper">` around our heading. We use `React.createElement` to render a `div` with a `className` of `'wrapper'`, and then pass our heading in as the child:

```
React.createElement('div', { className: 'wrapper' },
  React.createElement('h1', null, 'Hello World')
)
```

This will render this HTML:

```
<div class="wrapper">
  <h1>Hello World</h1>
</div>
```

Let's look at the working example.

HTML CSS JavaScript

```javascript
ReactDOM.render(
  React.createElement('div', { className: 'wrapper' },
    React.createElement('h1', { className: 'heading' }, 'Hello World!')
  ),
  document.getElementById('container')
);
```

output

Hello World!

This `.wrapper` div might have a `max-width` and other styling associated with it. By reusing this element, we make sure our application is consistent since it has the same styling everywhere! This is what *components* are for.

React components

What are components in React and how can we create them?

To create a `ReactComponent` we write a function that returns a `ReactElement`:

```
var Wrapper = function(props) {
  return React.createElement('div', { className: 'wrapper' });
}
```

We can then use these components like any DOM Node by passing it into a `createElement` call:

```
React.createElement(Wrapper);
// -> <div class="wrapper"></div>
```

Our wrapper component isn't very useful so far, since it doesn't render its children:

```
React.createElement(Wrapper, {}, 'Hello World!');
// -> <div class="wrapper"></div> (😢 no "Hello World" visible!)
```

This is because our component function gets passed the properties (`props`). In the last example we didn't use the properties we got passed from the `createElement(Wrapper)` call at all!

Let's render the children that our `Wrapper` gets passed:

```
var Wrapper = function(props) {
  // Render the children we get passed from the createElement(Wrapper) cal
l
  return React.createElement('div', { className: 'wrapper' }, props.childr
en);
}
```

Now the above example works perfectly:

```
React.createElement(Wrapper, {}, 'Hello World!');
// -> <div class="wrapper">Hello World</div>
```

Let's try rendering our heading inside our `Wrapper` component:

HTML CSS JavaScript

```javascript
var Wrapper = function(props) {
  return (
    React.createElement('div', { className: 'wrapper' }, props.children)
  );
};

ReactDOM.render(
  React.createElement(Wrapper, null,
    React.createElement('h1', { className: 'heading' }, 'Hello World!')
  ),
  document.getElementById('container')
);
```

output

Hello World!

Amazing! 🎉 Now we have a reusable `Wrapper` component that could make sure every page across our application has consistent styling.

Introducing JSX

JSX is the markup language that makes it easier to create React components

WE'LL COVER THE FOLLOWING

- JSX

JSX

You might have seen React code samples floating around, and something that might've struck you is the weird HTML-ish syntax in the JavaScript code that is used by most of the community.

This syntactic sugar is called "JSX", and is nothing but a wrapper for `React.createElement`!

Instead of calling `React.createElement` manually, we can use JSX to make the code look more like the rendered HTML:

```
<Wrapper>
  <h1 className="heading">Hello World</h1>
</Wrapper>
```

is the same thing as

```
React.createElement(Wrapper, null,
  React.createElement('h1', {className: 'heading'}, 'Hello World')
)
```

> Using JSX is a bit tricky: since it's a non-standard extension of JavaScript no browser will understand it. This means we have to *transpile* (compile JavaScript to JavaScript) our code with a build tool – thankfully, `Educative's JS REPL` does that automatically for us, so we don't have to worry about it.

worry about it.

Passing properties to our components is as easy as writing them as attributes on these HTML-like tags, and to add children we simply wrap them! The nice thing about JSX is that we can use JavaScript code in JSX by wrapping it in curly braces.

Let's convert our `Wrapper` component to use JSX:

HTML CSS JavaScript

javascript

```javascript
var Wrapper = function(props) {
  return (
    <div className="wrapper">{ props.children }</div>
  );
};

ReactDOM.render(
  <Wrapper>
    <h1 className="heading">Hello World!</h1>
  </Wrapper>,
  document.getElementById('container')
);
```

output

Hello World!

It's not that different from calling `createElement` manually, but JSX is much nicer to read and understand!

> JSX is the preferred way of writing react applications because it is easier to read and understand. Thus, this tutorial will from now on use JSX.

Weather app in React

Starting to write our first React.js application, we learn how to structure our app and how to fetch data from an API!

Let's use our knowledge to write an actual app! What we'll build is a weather application that'll display the current conditions and a 7 day forecast. Again, here is a live preview of what we're going to build:

Let's use the knowledge that we gained in the first part of this course and create a barebones *Hello World Weather app* that displays a heading saying

create a barebones *Hello World Weather app* that displays a heading saying "Weather"!

As we are now going to write a larger App, we are also going to split our code into multiple files. Now ***index.js*** *will be the entry point and include the logic for rendering the App component. We will start writing our own code in* ***App.js***

We'll need a bit of styling to make sure our app looks good. We've prepared that for you so you can focus on React. Just view the styling in the above example if you are interested.

We'll also need to be able to tell our app for which location we want the weather, so let's add a form with an input field and label that says "City, Country"!

> We nest the `input` inside the `label` so the input is focussed when users click on the label!

When entering something into the input field and pressing "Enter", the page refreshes and nothing happens. What we really want to do is fetch the data when a city and a country are input. Let's add an `onSubmit` handler to the `form` and a `fetchData` function to our component!

If you enter a City/Country name in the above form and press Enter, you will see the console.log statement in your browser console

By running `evt.preventDefault()` in fetchData (which is called when we press enter in the form), we tell the browser to not refresh the page and instead ignore whatever it wanted to and do what we tell it to. Right now, it logs "fetch weather data!" in the console over and over again whenever you submit the form. How do we get the entered city and country in that function though?

By storing the value of the text input in our local component state, we can grab it from that method. When we do that, we make our input a so-called

“controlled input”.

We’ll store the currently entered location in `this.state.location`, and add a utility method to our component called `changeLocation` that is called `onChange` of the text input and sets the state to the current text:

```
class App extends React.Component {
  fetchData = (evt) => { /* … */ };

  changeLocation = (evt) => {
    this.setState({
      location: evt.target.value
    });
  };

  render() {
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>I want to know the weather for
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.state.location}
              onChange={this.changeLocation}
            />
          </label>
        </form>
      </div>
    );
  }
}
```

As mentioned in Chapter 1, when saving anything to our local state, we have to predefine it. Let’s do that:

```
class App extends React.Component {
  state = {
    location: ''
  };

  ...
}
```

In our fetchData function, we can then access `this.state.location` to get the current location:

```
class App extends React.Component {
  state = { /* … */ };

  fetchData = (evt) => {
    evt.preventDefault();
    console.log('fetch data for', this.state.location);
  };

  changeLocation = (evt) => { /* … */ };

  render() { /* … */ }
}
```

Here’s the full working example of what we’ve learned so far

Now, whichever location you enter it should log "fetch data for <YourCity>, <YourCountry>!

Fetching data

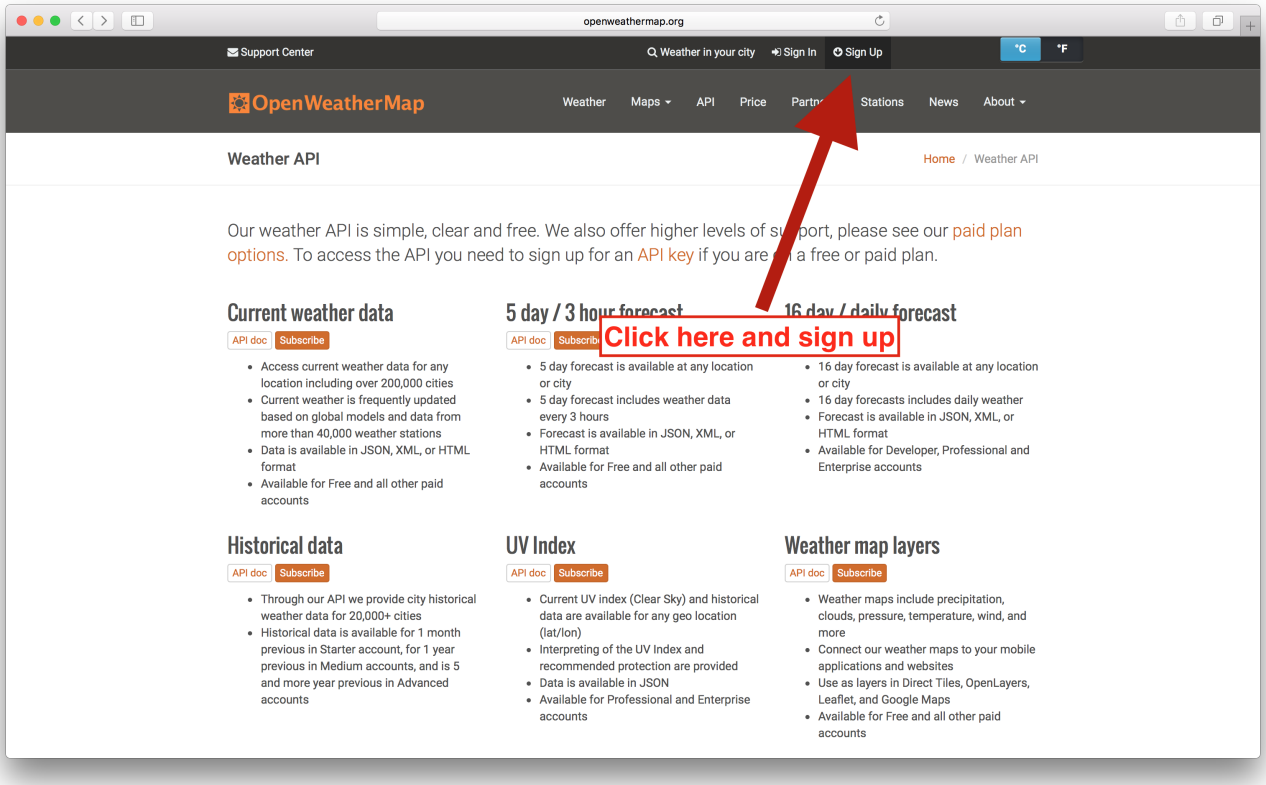
How are we going to fetch data for our weather app using React.js

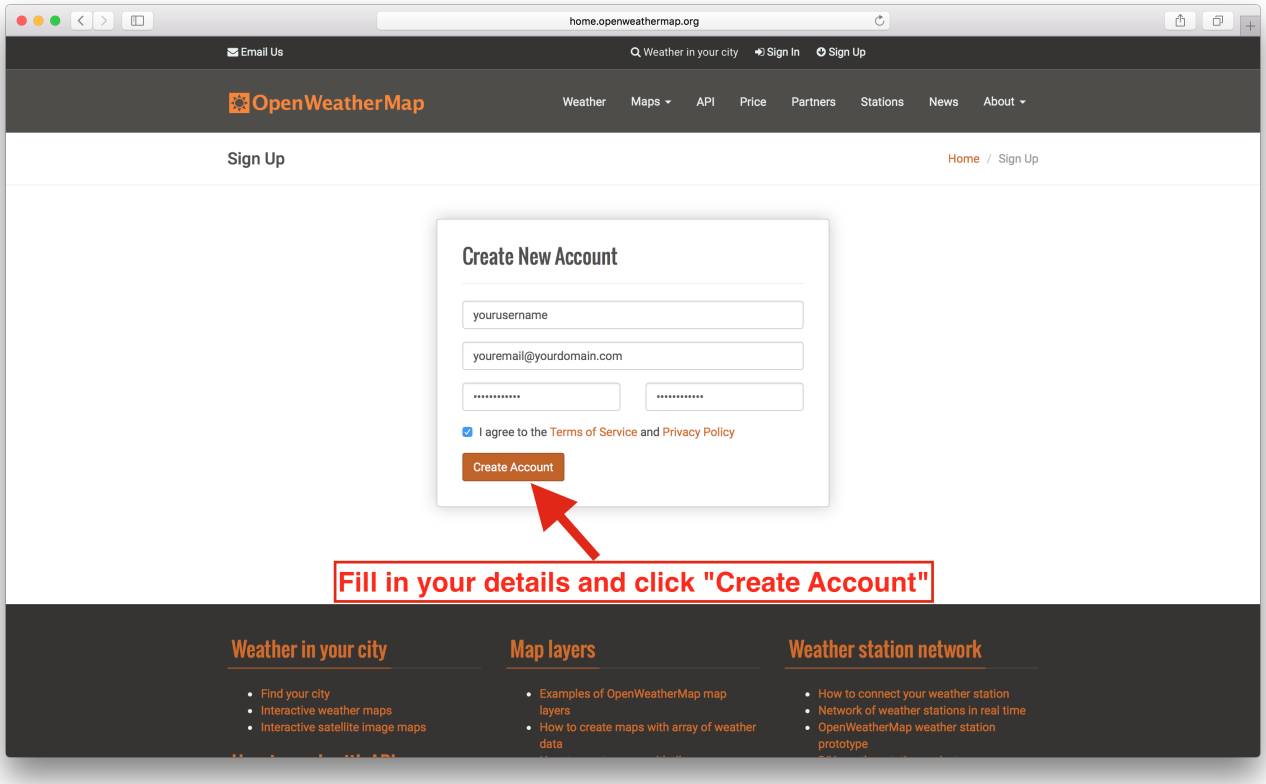
WE'LL COVER THE FOLLOWING

- Setting OpenWeatherMap account
- npm module: xhr
- Parsing data from OpenWeatherMap
- Creating the Url that fetches weather data
- Show current temperature
- Summary
- Additional Material

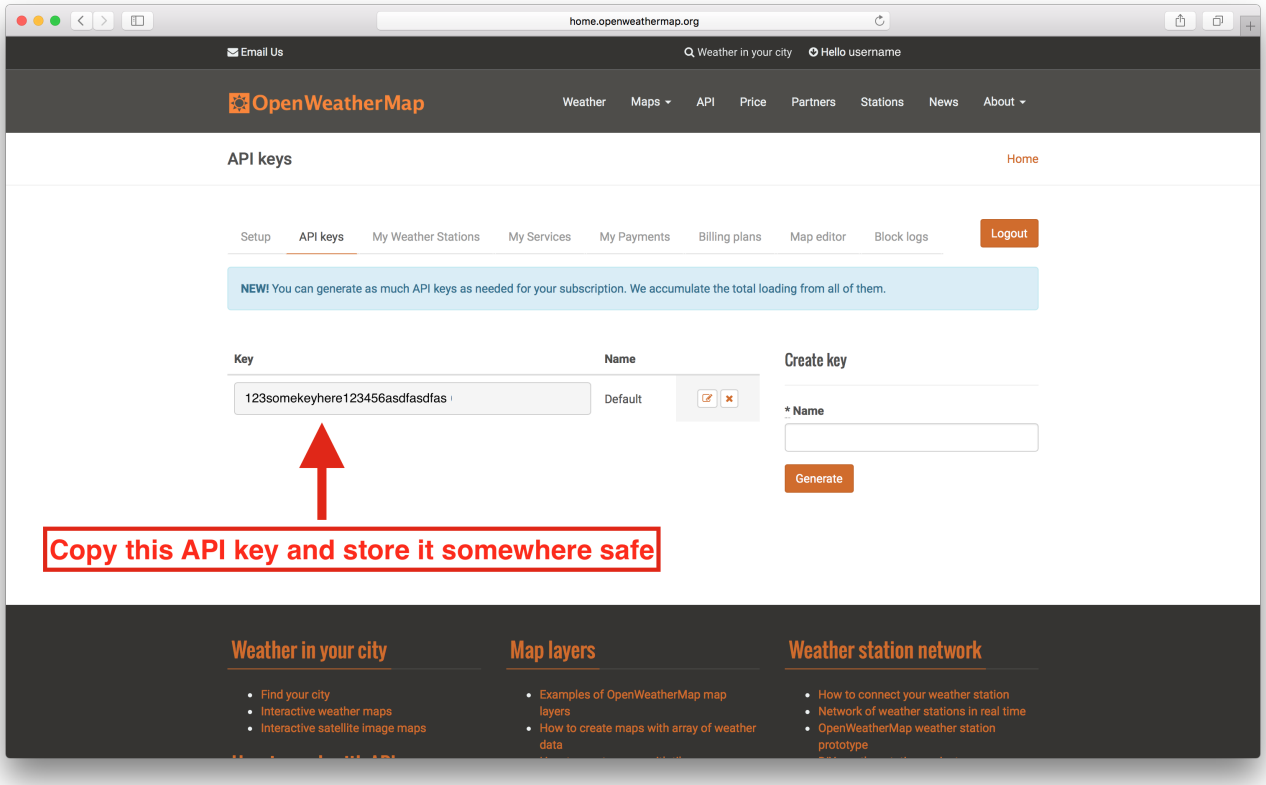
Setting OpenWeatherMap account

Let's get into fetching data. Instead of console logging a text, we need to get some weather information. We'll be using the OpenWeatherMap API for this task, which is a free service that provides access to data for basically all locations all around the world. You'll need to get an API key from it, so head over to openweathermap.org/api, press "Sign Up" in the top bar and register for a free account:





As soon as you've done that go to your API key page by going to home.openweathermap.org/api_keys, copy the `API key` from there and keep it somewhere safe.



Go ahead, signup. I'll wait for you here.

Now that we have access to all the weather data our heart could desire, let's get on with our app!

npm module: xhr

Inside our `fetchData` function, we'll have to make a request to the API. I like to use a npm module called `xhr` for this, a wrapper around the JavaScript XMLHttpRequest that makes said requests a lot easier

The general usage of `xhr` looks like this:

```
xhr({
  url: 'someURL'
}, function (err, data) {
  /* Called when the request is finished */
});
```

Parsing data from OpenWeatherMap

To get the weather forecast data, the structure of the URL we'll request looks like this:

```
http://api.openweathermap.org/data/2.5/forecast?q=CITY,COUNTRY&APPID=YOURAPIKEY&units=metric
```

Replace `CITY,COUNTRY` with a city and country combination of choice, replace `YOURAPIKEY` with your copied API key and open that URL in your browser. (it should look something like this: `http://api.openweathermap.org/data/2.5/forecast?q=Vienna,Austria&APPID=asdf123&units=metric`)

What you'll get is a JSON object that has the following structure:

```
"city": {
  "id": 2761369,
  "name": "Vienna",
  "coord": {
    "lon": 16.37208,
    "lat": 48.208488
  },
  "country": "AT",
  "population": 0,
  "sys": {
    "population": 0
  }
},
"cod": "200",
"message": 0.0046,
"cnt": 40,
"list": [ /* Hundreds of objects here */ ]
```

The top level `list` array contains time sorted weather data reaching forward 5 days. One of those weather objects looks like this: (only relevant lines shown)

```
{
  "dt": 1460235600,
  "main": {
    "temp": 6.94,
    "temp_min": 6.4,
    "temp_max": 6.94
  },
  "weather": [
    {
      "main": "Rain",
      /* …more data here */
    }
```

```
    }
  ],

  /* …more data here */
}
```

The five properties we care about are:

1. `dt_txt` - the time of the weather prediction.
2. `temp` - the expected temperature.
3. `temp_min` - the minimum expected temperature.
4. `temp_max` - the maximum expected temperature.
5. `weather[0].main` - a string description of the weather at that time.

OpenWeatherMap gives us a lot more data than that though, and I encourage you to snoop around a bit more and see what you could use to make the application more comprehensive!

Creating the Url that fetches weather data

As you can see, everything we really need to take care of is constructing the url and saving the returned data somewhere!

We know that the URL has a prefix that's always the same, `http://api.openweathermap.org/data/2.5/forecast?q=`, and a suffix that's always the same, `&APPID=YOURAPIKEY&units=metric`. The sole thing we need to do is insert the location the user entered into the URL!

> You can also change the units you get back by setting `units` to `imperial`: `http://api.openweathermap.org/data/2.5/forecast?q=something&APPID=YOURAPIKEY&units=imperial`

Now, if you're thinking this through you know what might happen – the user might enter spaces in the input! URLs with spaces aren't valid, so it wouldn't work and everything would break! While that is true, JavaScript gives us a very handy method to escape non-URL-friendly characters. It is called `encodeURIComponent()`, and this is how one uses it:

```
encodeURIComponent('My string with spaces'); // -> 'My%20string%20with%20spaces'
```

Show current temperature

We are going to modify our app to show the current temperature of the location that the user specifies.

Let's start writing some code

First, let's encode the location from the state and construct the URL that we need using that escaped location (*notice how we have a variable to assign the API key when the time comes*).

```
const API_KEY = "";
class App extends React.Component {
  state = { /* … */ };

  fetchData = (evt) => {
    evt.preventDefault();
    let location = encodeURIComponent(this.state.location);

    let location = encodeURIComponent(this.state.location);
    let urlPrefix = 'http://api.openweathermap.org/data/2.5/forecast?q=';
    let urlSuffix = '&APPID=' + API_KEY + '&units=metric';
    let url = urlPrefix + location + urlSuffix;
  };

  changeLocation = (evt) => { /* … */ };

  render() { /* … */ }
}
```

The last thing we need to do to get the data from the server by calling `fetch` with that url!

```
class App extends React.Component {
  fetchData = (evt) => {
        ...

    let url = urlPrefix + location + urlSuffix;

    fetch(url).then(r => r.json())
          .then(json => this.setState({
        data: json
        }))
        .catch(e => console.log("error"))
```

```
  };


  changeLocation = (evt) => { /* … */ };

  render() { /* … */ }
}
```

Now that we've got the weather data for the location we want in our component state, we can use it in our render method! Remember, the data for the current weather is in the `list` array, sorted by time. The first element of said array is thus the current temperature, so let's try to render that first. Here's how our render method will look:

```
render() {
    let currentTemp = 'Specify a location';
    if (this.state.data.list) {
      currentTemp = this.state.data.list[0].main.temp;
    }
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>I want to know the weather for
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.state.location}
              onChange={this.changeLocation}
            />
          </label>
        </form>
        <p className="temp-wrapper">
          <span className="temp">{ currentTemp }</span>
          <span className="temp-symbol">°C</span>
        </p>
      </div>
    );
  }
```

Now it's time to use your `OpenWeatherMAP API key`. Assign your key to the variable API_KEY in the example below and then enter a location and you'll see the current temperature! 🎉 Awesome!).

```
import React from 'react';
import './App.css';
import xhr from 'xhr';

const API_KEY = "82f40c24bce69950c7aa3d09e07b391b";

class App extends React.Component {
  state = {
      location: '',
      data: {}
  };

  fetchData = (evt) => {
    evt.preventDefault();

    if (!API_KEY) {
      console.log('Enter your API_KEY and the enter location');
      return;
    }

    let location = encodeURIComponent(this.state.location);
    let urlPrefix = '/cors/http://api.openweathermap.org/data/2.5/forecast?q=';
    let urlSuffix = '&APPID=' + API_KEY + '&units=metric';
    let url = urlPrefix + location + urlSuffix;

    xhr({
      url: url
    }, (err, data) => {
      if (err) {
        console.log('Error:', err);
        return;
      }

      this.setState({
        data: JSON.parse(data.body)
      });
    });
  };

  changeLocation = (evt) => {
    this.setState({
      location: evt.target.value
    });
  };

  render() {
    let currentTemp = 'Specify a location';
    if (this.state.data.list) {
      currentTemp = this.state.data.list[0].main.temp;
    }
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>I want to know the weather for
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.state.location}
              onChange={this.changeLocation}
            />
          </label>
```

```
        </form>
        <p className="temp-wrapper">
          <span className="temp">{ currentTemp }</span>
          <span className="temp-symbol">°C</span>
        </p>
      </div>
    );
  }
}

export default App;
```

Summary

We learned how to structure our application and then we created a controlled text input and used that to fetch our first data using `fetch`!

Now that we have the current temperature, we need to render the 5 day forecast! Continue with Chapter 3: Adding a Forecast Graph where we'll learn how to draw a graph!

Additional Material

- Official create-react-app docs
- What APIs Are And Why They're Important

Getting started with Plotly.js

In this lesson, we'll learn the basics of drawing charts with Plotly.js

WE'LL COVER THE FOLLOWING

- Plotly

Often, in real world applications, there's some third party library that needs to be used, or maybe some legacy code that needs to be bridged. To showcase how one would go about doing that, we'll be using Plotly to draw a beautiful temperature forecast graph!

Plotly

We'll explore the Plotly.js API and draw a scatter chart.

To use `plotly.js`, we need to add it to our application. Copy and paste the following line into your HTML:

```
<script src="https://cdn.plot.ly/plotly-1.8.0.min.js"></script>
```

Including this script gives us access to the `Plotly` variable in our code. Using `Plotly.newPlot`, we can easily create graphs to showcase the weather data:

```
Plotly.newPlot();
```

Let's look at the `newPlot` function in a little bit more detail:

1. The first argument is the `id` of the DOM element we want to render our plot into.
2. The second argument is an array of objects with a few properties for our plot, with `x`, `y` and `type` being the most relevant for us.

Plotly.js makes it easy to create a wide variety of plots, the one we care about

the most at the moment is a `scatter` plot. See the documentation here for

some examples.

This is what our `Plotly.newPlot` call will look like:

```
Plotly.newPlot('someDOMElementId', [{
  x: ourXAxisData,
  y: ourYAxisData,
  type: 'scatter'
}], {
  margin: {
    t: 0, r: 0, l: 30
  },
  xaxis: {
    gridcolor: 'transparent'
  }
}, {
  displayModeBar: false
});
```

As you can see, we also pass in some styling information as the third argument (we specify a few margins and hide the xaxis grid lines), and some options as the fourth argument. (we hide the mode bar).

Plotly.js has tons of options, I encourage you to check out the excellent documentation and play around with a few of them!

For getting our feet wet, we'll draw a chart of 9 numbers [-4, 4], their squares and their cubes. Here's the data that we will use for our chart:

`x-axis`: [-4, -3, -2, -1, 0, 1, 2, 3, 4]

`y-axis-1`: [16, 9, 4, 1, 0, 1, 4, 9, 16];

`y-axis-2`: [-64, -27, -8, -1, 0, 1, 8,27, 64];

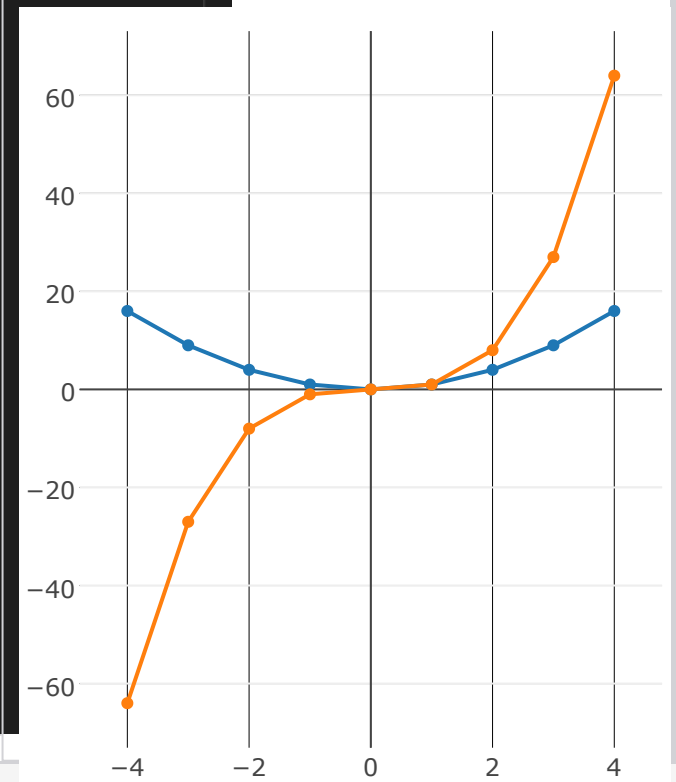
HTML JavaScript Output

```javascript
let numbers = [-4, -3, -2, 1, 2, 3, 4];
let squares = [16, 9, 4, 1, 0, 1, 4, 9, 16];
let cubes = [-64, -27, -8, -1, 0, 1, 8,27, 64];

Plotly.newPlot('content', [{
    name: 'squares',
    x: numbers,
```

output

```
    x: numbers,
    y: squares,
    type: 'scatter'
    }, {
    name: 'cubes',
    x: numbers,
    y: cubes,
    type: 'scatter'
    }], {
    margin: {
      t: 0, r: 0, l: 30
    },
    xaxis: {
      gridcolor: 'transparent'
    }
  }, {
    displayModeBar: false
});
```



As you can see, it's perfect for a weather forecast. But we are in the React world where everything is (or at least should be) a ***component***. You might have already guessed it - it's time to build our `Plot` component.

Creating a Plot component

We'll use our knowledge of React.js and Plotly.js to create a plot component that will show the weather forecast.

WE'LL COVER THE FOLLOWING

- Plot component
- Passing data to the Plot component

Plot component

To show our plot on the webpage, we will create a component which we'll call `Plot` (what a surprise!) so lets start by rendering just a div:

```
class Plot extends React.Component {
  render() {
    return (
      <div id="plot"></div>
    );
  }
}
```

> As you can see, I've added a `div` with an ID of `plot` above. This is the DOM element we'll reference in our `Plotly.newPlot` call!

Now, the problem we have here is that if we called `Plotly.newPlot` in our `render` method, it would be called over and over again, possibly multiple times per second! That's not optimal, we really want to call it once when we get the data and leave it be afterwards – how can we do that?

Thankfully, React gives us a lifecycle method called `componentDidMount`. It is called once when the component was first rendered, and never afterwards; perfect for our needs! Let's create a `componentDidMount` method and call `Plotly.newPlot` in there and pass it the ID of our `div`, `plot`, as the first

`Plotly.newPlot` in there and pass it the ID of our `div`, `plot`, as the first argument:

```
class Plot extends React.Component {
  componentDidMount() {
    Plotly.newPlot('plot');
  }

  render() {
    return (
      <div id="plot"></div>
    );
  }
}
```

Passing data to the Plot component

That alone won't do much though, we need to give it data too!

Let's modify our Plot component to accept following three props:

1. `xData` - an array containing numbers on x-axis.
2. `yData` - an array containing nmbers on y-axis.
3. `type` - which defines the type of the chart

Let's update our Plot component to receive data as props and call `Plotly.newPlot` to create the chart:

```
class Plot extends React.Component {
  componentDidMount() {
    Plotly.newPlot('plot', [{
      x: this.props.xData,
      y: this.props.yData,
      type: this.props.type
    }], {
      margin: {
        t: 0, r: 0, l: 30
      },
      xaxis: {
        gridcolor: 'transparent'
      }
    }, {
      displayModeBar: false
    });
  }
```

```
  render() {

    return (
      <div id="plot"></div>
    );
  }
}
```

Let's use the above `Plot` component to create a chart of squares of first 5 numbers. Here's how we are going to render it.

```
ReactDOM.render(
  <Plot
    xData={[1,2,3,4,5]}
    yData={[1, 4, 9, 16, 25]}
    type='scatter' />,
  document.getElementById('content')
);
```

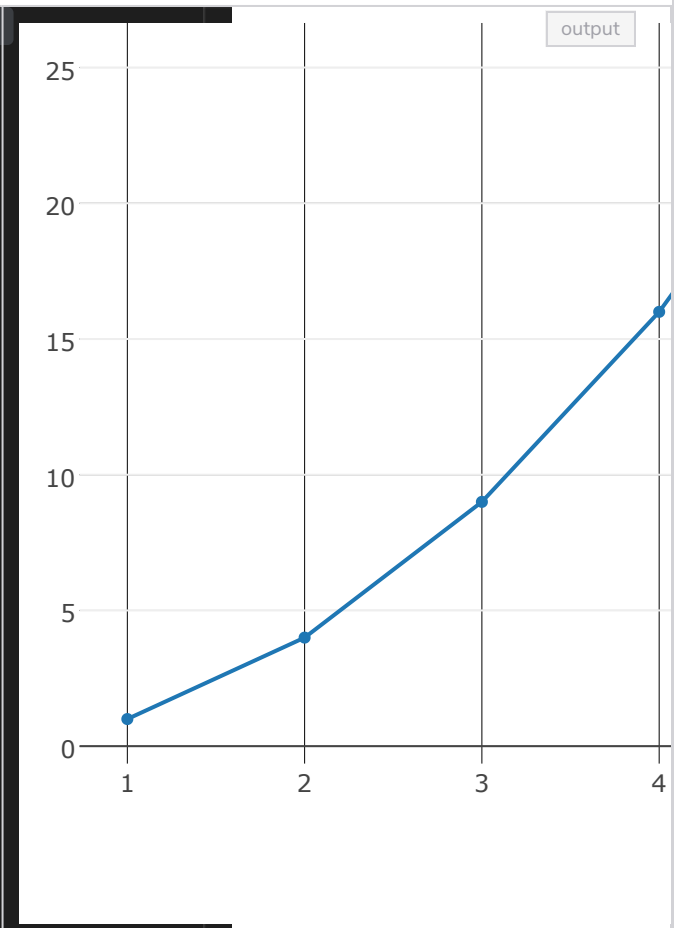
and finally, here's our working example:

HTML JavaScript Output

javascript

```
1  class Plot extends React.C{
2    componentDidMount() {
3      Plotly.newPlot('plot', [{
4        x: this.props.xData,
5        y: this.props.yData,
6        type: this.props.type
7      }], {
8        margin: {
9          t: 0, r: 0, l: 30
10       },
11       xaxis: {
12         gridcolor: 'transparent'
13       }
14     }, {
15       displayModeBar: false
16     });
17   }
18
19   render() {
20     return (
21       <div id="plot"></div>
22     );
23   }
24 }
25
26 ReactDOM.render(
27   <Plot
```

output



```
    xData={[1,2,3,4,5]}
    yData={[1, 4, 9, 16, 25]}
    type='scatter' />,
  document.getElementById('content')
```

We now have a `Plot` component that can take weather data as input and draw the forecast graph. We're getting closer to our forecast graph.

Creating forecast graph in weather app

We'll use our knowledge of React.js and Plotly.js to create a plot component that will show the weather forecast.

Now that we've our Plot component ready, we just need to pass in the weather data and we are done.

The problem is that we need the data for the x-axis and the y-axis to be separate, but the data we get from the OpenWeatherMap API doesn't make that distinction. This means we need to shape our data a little bit to suit our needs. What we want is human readable dates on the x-axis, and the degrees at that time on the y-axis!

We'll shape the data into a form we can use it in and save both the raw and the formed data in our component state. Remember, this is what the raw data looks like:

```
"city": {
  "id": 2761369,
  "name": "Vienna",
  "coord": {
    "lon": 16.37208,
    "lat": 48.208488
  },
  "country": "AT",
  "population": 0,
  "sys": {
    "population": 0
  }
},
"cod": "200",
"message": 0.0046,
"cnt": 40,
"list": [ /* Hundreds of objects here */ ]
```

With the `list` array containing objects of this form:

```
{
  "dt_txt": "2016-04-09 18:00:00",

  "main": {
    "temp": 6.94,
    "temp_min": 6.4,
    "temp_max": 6.94
  },
  "weather": [
    {
      "main": "Rain",
      /* …more data here */
    }
  ],
  /* …more data here */
}
```

What we really care about is `data.list[element].dt_txt`, a human-readable timestamp, and `data.list[element].main.temp`, the temperature at that time.

We'll loop through all the weather information we have, making two arrays and fill one with the timestamps, and another array with the temperatures. *(We'll use the `push` method of arrays, which adds an element to the end of an array).*

We'll do that in the `fetchData` method, so we only recalculate the data when new one comes in and not on every render. (which would possibly mean shaping the data every second or more!) This is what happens when the data comes back in at the moment

Our `xhr` call to OpenWeatherMap will now look like this:

```
xhr({
      url: url
    }, () => (err, data) {
      var body = JSON.parse(data.body);
      var list = body.list;
      var dates = [];
      var temps = [];
      for (var i = 0; i < list.length; i++) {
        dates.push(list[i].dt_txt);
        temps.push(list[i].main.temp);
      }

      this.setState({
```

```
        data: body


      });
    });
```

We also need to add those new properties to our initial state:

```
class App extends React.Component {
  state = {
    location: '',
    data: {},
    dates: [],
    temps: []
  };

  fetchData = (evt) => { /* … */ };

  changeLocation = (evt) => { /* … */ };

  render() { /* … */ }
}
```

Now that we have that data saved in our component state, we can render our plot by passing `this.state.dates` as the x-axis data, `this.state.temps` as the y-axis data and we'll also pass it a `type` prop of `"scatter"`!

Here's the working example (*reminder: you will need to update your OpenWeatherMap API key to make it work*):

```
import React from 'react';
import './App.css';
import xhr from 'xhr';

import Plot from './Plot.js';

const API_KEY = "82f40c24bce69950c7aa3d09e07b391b";

class App extends React.Component {
  state = {
      location: '',
      data: {},
      dates: [],
      temps: []
  };

  fetchData = (evt) => {
    evt.preventDefault();

    if (!API_KEY) {
```

```
    if (!API_KEY) {
      console.log('Enter your API_KEY and the enter location');
      return;
    }

    let location = encodeURIComponent(this.state.location);
    let urlPrefix = '/cors/http://api.openweathermap.org/data/2.5/forecast?q=';
    let urlSuffix = '&APPID=' + API_KEY + '&units=metric';
    let url = urlPrefix + location + urlSuffix;

    xhr({
      url: url
    }, (err, data) => {
      if (err) {
        console.log('Error:', err);
        return;
      }
      var body = JSON.parse(data.body);
      var list = body.list;
      var dates = [];
      var temps = [];
      for (var i = 0; i < list.length; i++) {
        dates.push(list[i].dt_txt);
        temps.push(list[i].main.temp);
      }

      this.setState({
        data: body,
        dates: dates,
        temps: temps
      });
    });
  };

  changeLocation = (evt) => {
    this.setState({
      location: evt.target.value
    });
  };

  render() {
    let currentTemp = 'Specify a location';
    if (this.state.data.list) {
      currentTemp = this.state.data.list[0].main.temp;
    }
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>I want to know the weather for
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.state.location}
              onChange={this.changeLocation}
            />
          </label>
        </form>
        <p className="temp-wrapper">
          <span className="temp">{ currentTemp }</span>
          <span className="temp-symbol">°C</span>
        </p>
        <h2>Forecast</h2>
```

```
        <Plot
                xData={this.state.dates}
          yData={this.state.temps}
          type="scatter"
        />
      </div>
    );
  }
}

export default App;
```

If you tried playing with the example, you would have notiecd that there is one problem: When we change the city and refetch data, the graph doesn't update. This is the case because we're solely using the `componentDidMount` lifecycle method, which is only ever called once when the component mounts. We also need to draw the plot again when new data comes in, i.e. when the component did update! (*hinthint*)

As you might have guessed, we can use the `componentDidUpdate` lifecycle method of our `Plot` component to fix this. However, instead of copy and pasting the `Plotly.newPlot` call (which is identical) in both methods, we should factor that out into a `drawPlot` method and call `this.drawPlot` from `componentDidMount/Update`.

Here's the working example with the updated code (*reminder again: you will need to update your OpenWeatherMap API key to make it work*):

```
import React from 'react';

class Plot extends React.Component {
  drawPlot = () => {
    Plotly.newPlot('plot', [{
      x: this.props.xData,
      y: this.props.yData,
      type: this.props.type
    }], {
      margin: {
        t: 0, r: 0, l: 30
      },
      xaxis: {
        gridcolor: 'transparent'
      }
    }, {
      displayModeBar: false
    });
  }

  componentDidMount() {
    this.drawPlot();
  }
```

```
  componentDidUpdate() {
    this.drawPlot();
  }

  render() {
    return (
      <div id="plot"></div>
    );
  }
}


export default Plot;
```

Now try it! You'll see a beautiful 5 day weather forecast rendered like this. Normally, creating a graph like this manually would take ages, but Plotly.js makes it incredibly easy!

Plotly.js also provides many other features like handling clicks on the graphs. I think you might have guessed where this is going. We are going to show the temperature of the day/time where user clicks on the graph.

Handling clicks on our forecast graph

We'll learn how to handle clicks on a Plotly.js graph and how to update our weather app in React.js to show the temperature of the day/time where the user has clicked.

WE'LL COVER THE FOLLOWING

- Setting up the click handler
- Updating our app state to store click data
- Summary
- Additional Material

Let's add one more feature to our weather application. When clicking on a specific point of our graph, we want to show the user in text the temperature at that date!

Setting up the click handler

The first thing we need to do is add an event listener to our graph. Thankfully, Plotly gives us a handy `plotly_click` event to listen to, like so:

```
// Called when a plot inside the DOM element with the id "someID" is click
ed
document.getElementById('someID').on('plotly_click', function(data) {
  /* …do something here with the data… */
});
```

The nice thing about `plotly_click` is that it doesn't pass you the event, it passes you a very useful `data` object. We care about two particular properties of that `data` object:

```
{
  "points": [{
    "x": "2016-07-29 03",
    "y": 17.4,
```

```
    /* …more data here… */


  }]
}
```

These tell us which date was clicked on and what the relevant temperature was, exactly what we want! We'll pass a function down to the `Plot` component called `onPlotClick` that will get called when the `plotly_click` event is fired, i.e. when a point on our forecast is clicked on.

Let's start off by binding that event listener in our `Plot` component. In our `drawPlot` method bind the `plotly_click` event to `this.props.onPlotClick`!

```
// Plot.js

class Plot extends React.Component {
  drawPlot = () => {
    Plotly.newPlot( /* … */ );
    document.getElementById('plot').on('plotly_click', this.props.onPlotCl
ick);
  };

  componentDidMount() { /* … */ }
  componentDidUpdate() { /* … */ }
  render() { /* … */ }
}
```

Perfect, but running this will not work since we don't pass an `onPropClick` prop to `Plot`. Let's jump to our `App` component and change that. First, we pass an `onPlotClick` prop to our `Plot` component calling our `App` component's (currently missing) `this.onPropClick` method:

```
// App.js

class App extends React.Component {
  state = { /* … */ };
  fetchData = (evt) => { /* … */ };
  changeLocation = (evt) => { /* … */ };

  render() {
    /* … */
    return (
      { /* … */ }
```

```
      <Plot
        xData={this.state.dates}

        yData={this.state.temps}
        onPlotClick={this.onPlotClick}
        type="scatter"
      />
      { /* … */ }
    );
  }
}
```

Then we add a first version of the `onPlotClick` method to our `App` component where we only log out the passed `data`:

```
import React from 'react';

class Plot extends React.Component {
  drawPlot = () => {
    Plotly.newPlot('plot', [{
      x: this.props.xData,
      y: this.props.yData,
      type: this.props.type
    }], {
      margin: {
        t: 0, r: 0, l: 30
      },
      xaxis: {
        gridcolor: 'transparent'
      }
    }, {
      displayModeBar: false
    });

    document.getElementById('plot').on(
      'plotly_click', this.props.onPlotClick);
  }

  componentDidMount() {
    this.drawPlot();
  }

  componentDidUpdate() {
    this.drawPlot();
  }

  render() {
    return (
      <div id="plot"></div>
    );
  }
}


export default Plot;
```

Now try opening your application, select a city and, when the forecast has rendered, click on a specific data point in the plot. If you see an object logged in your console containing an array called `points`, you're golden!

Updating our app state to store click data

Instead of logging the data, we now want to save that data in our state. Let's add a new object to our initial state called `selected`, which contains a `date` and a `temp` field. The date field will be an empty string by default, and the temp `null`:

```
class App extends React.Component {
  state = {
    location: '',
    data: {},
    dates: [],
    temps: [],
    selected: {
      date: '',
      temp: null
    }
  };

    /* ... Rest of the component ... */
  }
}
```

Now, when our `onPlotClick` method is called we'll set the `selected.date` to `data.points[0].x`, and the the `selected.temp` to `data.points[0].x`:

```
onPlotClick = (data) => {
    if (data.points) {
      this.setState({
        selected: {
          date: data.points[0].x,
          temp: data.points[0].y
        }
      });
    }
  };
```

Now that we have the necessary data in our state, we need to do something with it! Let's render some text saying "The current temperature on some-date is some-temperature°C!" if we have a date selected, and otherwise show the current date. We thus need to adapt the `render` method of our `App` component to include that. We check if `this.state.selected.temp` exists (i.e. isn't `null`, the default value), and if it does we render the text with `this.state.selected`:

```
    var currentTemp = 'not loaded yet';
    if (this.state.data.list) {
      currentTemp = this.state.data.list[0].main.temp;
    }
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>I want to know the weather for
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.state.location}
              onChange={this.changeLocation}
            />
          </label>
        </form>
        {(this.state.data.list) ? (
          <div className="wrapper">
            {/* Render the current temperature if no specific date is sele
cted */}
            <p className="temp-wrapper">
              <span className="temp">
                { this.state.selected.temp ? this.state.selected.temp : cu
rrentTemp }
              </span>
              <span className="temp-symbol">°C</span>
              <span className="temp-date">
                { this.state.selected.temp ? this.state.selected.dat
e : ''}
              </span>
            </p>
            <h2>Forecast</h2>
            <Plot
```

```
              xData={this.state.dates}
              yData={this.state.temps}

              onPlotClick={this.onPlotClick}
              type="scatter"
            />
          </div>
        ) : null}

      </div>
    );
```

There is one small user experience improvement we could do. When switching to a new city, the text persists because `this.state.selected.temp` still references the old data—in reality want to show the current temperature though!

To fix this, we set `selected` back to the default values in our `fetchData` method when the request has returned data:

```
fetchData = (evt) => {
    /* ... Rest of the code */
  self.setState({
        data: body,
        dates: dates,
        temps: temps,
        selected: {
          date: '',
          temp: n
        }
      }}
```

Let's look at our app again.

```
import React from 'react';

class Plot extends React.Component {
  drawPlot = () => {
    Plotly.newPlot('plot', [{
      x: this.props.xData,
      y: this.props.yData,
      type: this.props.type
    }], {
      margin: {
        t: 0, r: 0, l: 30
      },
      xaxis: {
        gridcolor: 'transparent'
```

```
      }
    }, {
      displayModeBar: false
    });

    document.getElementById('plot').on(
      'plotly_click', this.props.onPlotClick);
  }

  componentDidMount() {
    this.drawPlot();
  }

  componentDidUpdate() {
    this.drawPlot();
  }

  render() {
    return (
      <div id="plot"></div>
    );
  }
}


export default Plot;
```

Summary

We've created a new `Plot` component, shaped the data we get from the OpenWeatherMap API to suit our needs and used Plotly.js to render a beautiful and interactive 5 day weather forecast!

Additional Material

- Official plotly.js docs
- OpenWeatherMap API
- JavaScript Graphing Library Comparison

React state management: Flux and Redux

Learn how to handle application state in a centralized store with Redux, the most popular unidirectional data flow library!

WE'LL COVER THE FOLLOWING

- The Concept

We've been using a top-level component called `App` to manage our global application state. That works fine for a small application, but as we add more and more functionality it becomes very tedious to work with.

Together with React, Facebook released something called Flux. Flux is a methodology that helps you manage your global application state. Flux works fine, but has the downside that it uses events, which can lead to quite a bit of confusion.

Thankfully, Dan Abramov stepped in and created Redux. Redux has the same core concept as Flux, but works without events, is much easer to understand and now basically the standard for application state management.

> Note: None of the following concepts and technologies are **necessary** to build a production app with react. Many people use them to their advantage, but they have some downsides too. (which we'll examine)

The Concept

Remember the initial state of our `App` component? It looks like this:

```
state = {
  return {
    location: '',
    data: {},
```

```
    dates: [],
    temps: [],

    selected: {
      date: '',
      temp: null
    }
  };
};
```

The object we return from this function is our entire application state. At the first start of our application, our state thus looks like this:

```
{
  location: '',
  data: {},
  dates: [],
  temps: [],
  selected: {
    date: '',
    temp: null
  }
}
```

When users now change the location input field, the `location` field of our state changes:

```
{
  location: 'Vienna, Austria',
  data: {},
  dates: [],
  temps: [],
  selected: {
    date: '',
    temp: null
  }
}
```

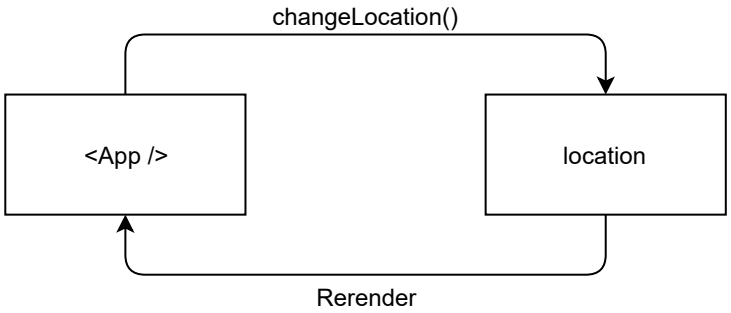
Instead of directly changing that location with `this.setState`, we'll call a function called `changeLocation` from our component. Redux will pick up that said function was called, do its magic and change the `location` field of our application state.

Now that the location is different and thus our application state has changed,

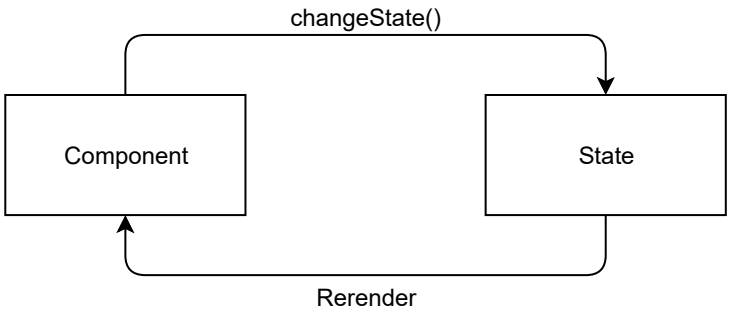
our main `<App />` component will automatically rerender with the new data! (just like with component state)

The big advantage of this approach is that the component no longer needs to know how exactly we save the location. We could be persisting it as a coordinate, we could save it without whitespace, but the component doesn't have to care about that—the component only calls `changeLocation` and that's it! The application state is thus decoupled from the individual components.

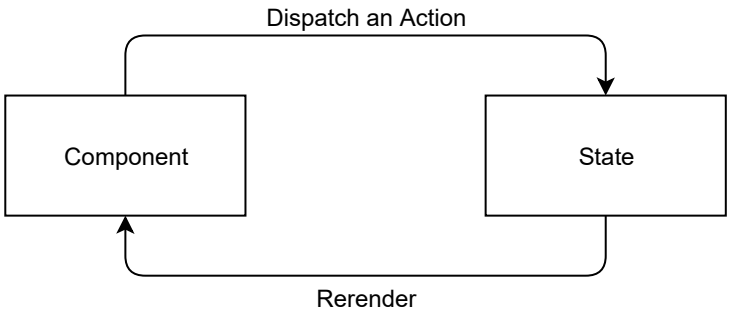
This cycle of state management thus looks like this:



If we put this into more general terms, we call a function which changes something in the application state which rerenders some component:



We'll now need to introduce some terminology before we can finally start implementing this. This function that we call to change the application state is called an "action" in Redux, and we "dispatch" the "action". Let's change the cycle one last time with the terminology:



Writing our first Redux parts

Let's take the first steps in using Redux for our weather app

WE'LL COVER THE FOLLOWING

- Our First Action
- Tying it all together
  - Creating Actions
  - Creating Reducers
  - Enabling Store to use Reducers
  - Connecting App to global redux state
  - mapStateToProps
- Wiring up the rest

Our First Action

Let's write our first action! We'll start with the location field, since it's a very typical example. An action function in Redux returns an object with a `type` and can optionally also pass some data along the way. Our `changeLocation` action looks like this:

```
function changeLocation(location) {
  return {
    type: 'CHANGE_LOCATION',
    location: location
  };
}
```

This action thus has a type of `'CHANGE_LOCATION'` and passes along some data with the `location` property.

That's nice and all, but this won't change the state automatically. We have to tell Redux what to do when this action comes in, which we do in a so-called

tell Redux what to do when this action comes in, which we do in a so-called reducer.

A reducer is a simple function that takes two arguments, the current state and the action that was dispatched:

```
function mainReducer(state, action) {
  return state;
}
```

Right now, no matter what action comes in and what data it has the state will always stay the same – that's not quite optimal, as nobody will be able to work with the app! Let's change the `location` field in the state based on the data in the action with the `'CHANGE_LOCATION'` type.

```
function mainReducer(state, action) {
  switch (action.type) {
    case 'CHANGE_LOCATION':
      state.location = action.location;
      return state;
  }
}
```

What we're doing here is *mutating* the state. We assign `state.location` the value of `action.location`. This is discouraged by Redux because it introduces potential bugs and side effects. What we instead should be doing is *returning a new object* which is a copy of the state!

JavaScript has a handy function called `Object.assign`, which allows you to do that. Let's take a look at the solution first:

```
function mainReducer(state, action) {
  switch (action.type) {
    case 'CHANGE_LOCATION':
      return Object.assign({}, state, {
        location: action.location
      });
  }
}
```

By passing in a new, empty object (`{}`) as the first argument and the current `state` as the second one, we create a carbon copy of the state. The third argument of the function (`{ location: action.location }`) is *just the changes*

argument of the function (`{ location: action.location }`) is *just the changes to our state*!

This creates a new object, meaning the state stays the same which is A+ behaviour and will keep us from a lot of bugs!

With a bit of glue this'll already work! We should do two more small things to make this better: we should return the state unchanged if no action we want to handle comes in and we should use the initial state if state is undefined:

```
var initialState = {
  location: '',
  data: {},
  dates: [],
  temps: [],
  selected: {
    date: '',
    temp: null
  }
};

function mainReducer(state = initialState, action) {
  switch (action.type) {
    case 'CHANGE_LOCATION':
      return Object.assign({}, state, {
        location: action.location
      });
    default:
      return state;
  }
}
```

We'll now need to `dispatch` this action when the location changes:

```
class App extends React.Component {
 fetchData = (evt) => { /* … */ };
 onPlotClick = (data) =>  { /* … */ };
 changeLocation = (evt) => {
   this.props.dispatch(changeLocation(evt.target.value));
 };
 render() { /* … */ }
});
```

> Don't worry about where `this.props.dispatch` comes from for now, we'll

> get to that!

Imagine `evt.target.value` is `"Sydney, Australia"`, this is what our global state is going to look like when we `dispatch` the `changeLocation` action:

```
{
  location: 'Sydney, Australia',
  /* …the rest stays the same… */
}
```

Tying it all together

Now that we understand the basic parts that are involved, let's tie it all together!

First, we need to create a store for our state and provide the state to our root `App` component. The `store` combines all of the apps reducers and (as the name suggests) stores the state. Once the store is set up though, you can forget about it again since we'll be using the state, but not the store directly!

> Redux allows us to create a single Store for the whole App

We do this in our main `index.js` file, and we'll use the `createStore` function from the `redux` package and the `Provider` component from the `react-redux` package.

First, `import` those functions:

```
// index.js

/* … */
import ReactDOM from 'react-dom';

import { createStore } from 'redux';

import { Provider } from 'react-redux';

import App from './App.js';
/* … */
```

Then we need to create our store:

Then we need to create our store:

```
// index.js

/* … */
import App from './App.js';

var store = createStore();

ReactDOM.render(
/* … */
);
```

Lastly, we need to wrap our `App` component in the `Provider` and pass in the store:

Lastly, we need to wrap our `App` component in the `Provider` and pass in the store:

```
/* … */
ReactDOM.render(
    <Provider store={store}>
        <App />
    </Provider>,
  document.getElementById('root')
);
/* … */
```

And that's it, our Redux integration is done! 🎉. Let's look at this in action

```
import ReactDOM from 'react-dom';
import { createStore } from 'redux';
import { Provider } from 'react-redux';

import App from './App';

let store = createStore();


ReactDOM.render(
  <Provider store={store}>
        <App />
  </Provider>,
  document.getElementById('root')
);
```

Creating Actions

Creating Actions

If you tried playing with the above example, you would have noticed that it doesn't do anything yet. Let's create an `actions.js` file and put our `changeLocation` action from above inside:

```
// actions.js

function changeLocation(location) {
  return {
    type: 'CHANGE_LOCATION',
    location: location
  };
}
```

We'll want to import it in other files, so we need to `export` it for that to work:

```
// actions.js

export function changeLocation(location) {
    return {
        type: 'CHANGE_LOCATION',
        location: location
    };
}
```

Awesome, we've got our first action – now we need to add our reducer!

Creating Reducers

Same deal as with the action, add a `reducers.js` file and export our previously written reducer from there:

```
// reducers.js

var initialState = {
  location: '',
  data: {},
  dates: [],
  temps: [],
  selected: {
    date: '',
    temp: null
  }
```

```
};

export default function mainReducer(state = initialState, action) {
  switch (action.type) {
    case 'CHANGE_LOCATION':
      return Object.assign({}, state, {
        location: action.location
      });
    default:
      return state;
  }
}
```

> We export the reducer by default since it'll be the only thing we're exporting from that file

That's our reducer done.

Enabling Store to use Reducers

Now we need to tell our store to use that reducer, so we `import` and pass it into the `createStore` call in the `index.js`:

```
// index.js

/* … */
import App from './App.js';

import mainReducer from './reducers';

var store = createStore(mainReducer);

ReactDOM.render(
/* … */
);
```

Connecting App to global redux state

Awesome, now everything's wired up except our `App` component! We need to connect it to the global redux state, which the `react-redux` module thankfully has a handy function for. Instead of exporting the raw `App` component, we export the `connect`ed component:

```
// App.js

/* …more imports… */

import { connect } from 'react-redux';

/* … */

export default connect()(App);
```

mapStateToProps

While this is nice, we also need to tell `connect` that it should inject the `location` field we have in our reducer into this component. We do this by passing in a function as the first argument that takes the entire state, and then we return what we want to inject as props into our component. (this automatically injects `dispatch` to run our actions, which is why we can use `this.props.dispatch` in the `App` component)

```
// App.js

/* … */

export default connect(function (state) {
    return {
        location: state.location
    };
})(App);
```

This function is called `mapStateToProps`, let's make that an external function so it's a bit clearer:

```
// App.js

/* … */

function mapStateToProps(state) {
    return {
        location: state.location
    };
}
```

```
export default connect(mapStateToProps)(App);
```

And that's everything need to get our App to get the location from the Redux state! Let's adapt our `App` to get the location from the props and see it in action

```
var initialState = {
  location: '',
  data: {},
  dates: [],
  temps: [],
  selected: {
    date: '',
    temp: null
  }
};

export default function mainReducer(state = initialState, action) {
  switch (action.type) {
    case 'CHANGE_LOCATION':
      return Object.assign({}, state, {
        location: action.location
      });
    default:
      return state;
  }
}
```

That's everything needed to get the initial wiring done! Open this in your browser and change the location input, you should see the value adjusting – this means redux is working as expected!

Wiring up the rest

Let's wire up some other actions, the goal here is to get rid of the entire component state of the `App` component! Let's take a look at the selected date and temperature. The first we'll write is two actions, `setSelectedDate` and `setSelectedTemp`, that pass on the value that they get passed in.

```
// actions.js

export function setSelectedDate(date) {
  return {
    type: 'SET_SELECTED_DATE',
    date: date
  };
}
```

```
export function setSelectedTemp(temp) {
  return {
    type: 'SET_SELECTED_TEMP',
    temp: temp
  };
}
```

Nothing fancy here, standard actions like the `changeLocation` one.

Let's add those two constants to our reducer, and also adjust the initial state a bit to include those fields:

```
// reducers.js
export default function mainReducer(state = initialState, action) {
  switch (action.type) {
    case 'CHANGE_LOCATION':
      return Object.assign({}, state, {
        location: action.location
      });
    case 'SET_SELECTED_TEMP':
      return state;
    case 'SET_SELECTED_DATE':
      return state;
    default:
      return state;
  }
}
```

Now our reducer just needs to return the changed state for those actions:

```
// reducers.js

export default function mainReducer(state = initialState, action) {
  switch (action.type) {
    case 'CHANGE_LOCATION':
      return Object.assign({}, state, {
        location: action.location
      });
    case 'SET_SELECTED_TEMP':
      return Object.assign({}, state, {
        selected: {
          temp: action.temp,
          date: state.selected.date
        }
      });
```

```
    });
    case 'SET_SELECTED_DATE':
      return Object.assign({}, state, {
        selected: {
          date: action.date,
          temp: state.selected.temp
        }
      });
    default:
      return state;
  }
}
```

Now let's wire it all up again in our `App` component. The below example has three more actions (and constants and reducer cases) that need to be implemented here: `setData`, `setDates` and `setTemps`. I'll leave it up to you here to implement them, taking inspiration from our already implemented actions!

When you are done (or feel stuck), move to the next lesson and see all the actions & reducers implemented.

```
import React from 'react';
import { connect } from 'react-redux';

import './App.css';
import xhr from 'xhr';

import Plot from './Plot.js';

import {
  changeLocation,
  setSelectedTemp,
  setSelectedDate
} from './actions';


const API_KEY = "82f40c24bce69950c7aa3d09e07b391b";

class App extends React.Component {
  state = {
      data: {},
      dates: [],
      temps: []
  };

  onPlotClick = (data) => {
    if (data.points) {
      let number = data.points[0].pointNumber;
      this.props.dispatch(setSelectedDate(data.points[0].x));
      this.props.dispatch(setSelectedTemp(data.points[0].y));
    }
  }
```

```
  };

  fetchData = (evt) => {

    evt.preventDefault();

    if (!API_KEY) {
      console.log('Enter your API_KEY and the enter location');
      return;
    }

    let location = encodeURIComponent(this.props.location);
    let urlPrefix = '/cors/http://api.openweathermap.org/data/2.5/forecast?q=';
    let urlSuffix = '&APPID=' + API_KEY + '&units=metric';
    let url = urlPrefix + location + urlSuffix;

    xhr({
      url: url
    }, (err, data) => {
      if (err) {
        console.log('Error:', err);
        return;
      }
      var body = JSON.parse(data.body);
      var list = body.list;
      var dates = [];
      var temps = [];
      for (var i = 0; i < list.length; i++) {
        dates.push(list[i].dt_txt);
        temps.push(list[i].main.temp);
      }

      this.setState({
        data: data,
        dates: dates,
        temps: temps,
      });

      this.props.dispatch(setSelectedTemp(null));
      this.props.dispatch(setSelectedDate(''));
    });
  };

  changeLocation = (evt) => {
    this.props.dispatch(changeLocation(evt.target.value));
  };

  render() {
    var currentTemp = 'not loaded yet';
    if (this.state.data.list) {
      currentTemp = this.state.data.list[0].main.temp;
    }
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>I want to know the weather for
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.props.location}
              onChange={this.changeLocation}
            />
```

```
          </label>
        </form>
        {(this.state.data.list) ? (
          <div className="wrapper">
            {/* Render the current temperature if no specific date is selected */}
            <p className="temp-wrapper">
              <span className="temp">
                { this.state.selected.temp ? this.state.selected.temp : currentTemp }
              </span>
              <span className="temp-symbol">°C</span>
              <span className="temp-date">
                <p>The temperature on { this.props.selected.date } will be { this.props.selec
              </span>
            </p>
            <h2>Forecast</h2>
            <Plot
              xData={this.state.dates}
              yData={this.state.temps}
              onPlotClick={this.onPlotClick}
              type="scatter"
            />
          </div>
        ) : null}

      </div>
    );
  }
}

function mapStateToProps(state) {
        return {
                location: state.location,
    selected: state.selected
        };
}

export default connect(mapStateToProps)(App);
```

Our completed Weather App with Redux

Let's look at Actions, Reducers and other parts of our weather app.

WE'LL COVER THE FOLLOWING

- Fetching Data: Going beyond xhr

Did you try implementing the Actions left out in the last lesson. If not, try it first.

Are you done? This is what your `App` component should look like now:

```
import React from 'react';
import './App.css';
import xhr from 'xhr';
import { connect } from 'react-redux';

import Plot from './Plot';
import {
  changeLocation,
  setData,
  setDates,
  setTemps,
  setSelectedDate,
  setSelectedTemp
} from './actions';

class App extends React.Component {
  fetchData = (evt) => {
    evt.preventDefault();

    var location = encodeURIComponent(this.props.location);

    var urlPrefix = '/cors/http://api.openweathermap.org/data/2.5/forecast?q=';
    var urlSuffix = '&APPID=dbe69e56e7ee5f981d76c3e77bbb45c0&units=metric';
    var url = urlPrefix + location + urlSuffix;

    var self = this;

    xhr({
      url: url
    }, function (err, data) {

      var body = JSON.parse(data.body);
      var list = body.list;
```

```
    var dates = [];
    var temps = [];
    for (var i = 0; i < list.length; i++) {
      dates.push(list[i].dt_txt);
      temps.push(list[i].main.temp);
    }

    self.props.dispatch(setData(body));
    self.props.dispatch(setDates(dates));
    self.props.dispatch(setTemps(temps));
    self.props.dispatch(setSelectedDate(''));
    self.props.dispatch(setSelectedTemp(null));
  });
};

onPlotClick = (data) => {
  if (data.points) {
    var number = data.points[0].pointNumber;
    this.props.dispatch(setSelectedDate(this.props.dates[number]));
    this.props.dispatch(setSelectedTemp(this.props.temps[number]))
  }
};

changeLocation = (evt) => {
  this.props.dispatch(changeLocation(evt.target.value));
};

render() {
  var currentTemp = 'not loaded yet';
  if (this.props.data.list) {
    currentTemp = this.props.data.list[0].main.temp;
  }
  return (
    <div>
      <h1>Weather</h1>
      <form onSubmit={this.fetchData}>
        <label>City, Country
          <input
            placeholder={"City, Country"}
            type="text"
            value={this.props.location}
            onChange={this.changeLocation}
          />
        </label>
      </form>
      {/*
        Render the current temperature and the forecast if we have data
        otherwise return null
      */}
      {(this.props.data.list) ? (
        <div>
          {/* Render the current temperature if no specific date is selected */}
          {(this.props.selected.temp) ? (
            <p>The temperature on { this.props.selected.date } will be { this.props.selecte
          ) : (
            <p>The current temperature is { currentTemp }°C!</p>
          )}
          <h2>Forecast</h2>
          <Plot
            xData={this.props.dates}
            yData={this.props.temps}
            onPlotClick={this.onPlotClick}
```

```
              type="scatter"
            />
          </div>
        ) : null}

      </div>
    );
  }
}

// Since we want to have the entire state anyway, we can simply return it as is!
function mapStateToProps(state) {
  return state;
}

export default connect(mapStateToProps)(App);
```

Fetching Data: Going beyond xhr

We’re almost done but we still have that ugly `xhr({})` call in our `fetchData` function though. This works, but as we add more and more components to our application it’ll become hard to figure out where what data is fetched.

That’s why the redux community has adopted `redux-thunk` as a new standard for fetching data!

Redux-Thunk: A better way to fetch data

Fetching data is Redux apps can be streamlined by using redux-thunk. We are going to use redux-thunk to fetch weather data

WE'LL COVER THE FOLLOWING

- First implementation
- Wiring it up
- Additional Material

The idea behind `redux-thunk` is that we return a function from an action that gets passed `dispatch`. This allows us to do asynchronous things (like data fetching) in our actions:

```
function someAction()
  // Notice how we return a function – this is what's called a "thunk"!
  return function thisIsAThunk(dispatch) {
    // Do something asynchronous in here
  }
}
```

First implementation

Let's try to write an action called `fetchData` that fetches our data! Start with the basic structure:

```
// actions.js

/* …more actions here… */

export function fetchData() {
  return function thunk(dispatch) {
    // LET'S FETCH OUR DATA HERE
  }
}
```

Now let's copy and paste the `xhr` call from the `App` component and put it in there:

```
// actions.js

/* …more actions here… */

export function fetchData() {
  return function thunk(dispatch) {
    xhr({
      url: url
    }, function (err, data) {

      var body = JSON.parse(data.body);
      var list = body.list;
      var dates = [];
      var temps = [];
      for (var i = 0; i < list.length; i++) {
        dates.push(list[i].dt_txt);
        temps.push(list[i].main.temp);
      }

      self.props.dispatch(setData(body));
      self.props.dispatch(setDates(dates));
      self.props.dispatch(setTemps(temps));
      self.props.dispatch(setSelectedDate(''));
      self.props.dispatch(setSelectedTemp(null));
    });
  }
}
```

Now we need to fix three things: 1) We need to import `xhr`, 2) we need to get the URL from the action and 3) we need to rename all `self.props.dispatch` calls to `dispatch`:

```
// actions.js

// REQUIRE xhr
import xhr from 'xhr';

/* …more actions here… */
```

```
// PASS URL IN HERE
export function fetchData(url) {

  return function thunk(dispatch) {
    xhr({
      url: url
    }, function (err, data) {

      var data = JSON.parse(data.body);
      var list = data.list;
      var dates = [];
      var temps = [];
      for (var i = 0; i < list.length; i++) {
        dates.push(list[i].dt_txt);
        temps.push(list[i].main.temp);
      }
      // RENAME self.props.dispatch TO dispatch
      dispatch(setData(data));
      dispatch(setDates(dates));
      dispatch(setTemps(temps));
      dispatch(setSelectedDate(''));
      dispatch(setSelectedTemp(null));
    });
  }
}
```

Well, that was easy! That's our thunked action done – let's call it from our `App` component:

```
/* … */

class App extends React.Component {
  fetchData = (evt) => {
    evt.preventDefault();

    var location = encodeURIComponent(this.props.location);

    var urlPrefix = 'http://api.openweathermap.org/data/2.5/forecast?q=';
    var urlSuffix = '&APPID=dbe69e56e7ee5f981d76c3e77bbb45c0&units=metri
c';
    var url = urlPrefix + location + urlSuffix;

    this.props.dispatch(fetchData(url));
  },
  onPlotClick = (data) => { /* … */ },
  changeLocation = (evt) => { /* … */ },
```

```
  render() { /* … */ }
});


/* … */
```

That makes our `App` so much nicer to work with already!

Wiring it up

The last step is wiring up `redux-thunk`. `redux-thunk` is a so-called "middleware". Middlewares sit in between the action and the reducers, every action you dispatch gets passed to all middlewares you add. (that's why they're called *middle* ware)!

Now let's `apply` the `thunk` middleware in our `createStore` call in `index.js`:

```
// index.js

/* … */
import { createStore, applyMiddleware } from 'redux';
import thunkMiddleware from 'redux-thunk';

/* … */

var store = createStore(
  mainReducer,
  applyMiddleware(thunkMiddleware)
);
/* … */
```

And that's it, everything should be working again now. Look how easy it is to handle our components, how nicely everything is separeted by concern and how easy it would be to add a new feature to our app! That's the power of redux, our application is easier to reason about and to handle, instead of having one massive top-level `App` component we separate the concerns properly.

Here's our Redux app in action

```
import React from 'react';
import './App.css';
import { connect } from 'react-redux';

import Plot from './Plot';
```

```
import {
  changeLocation,
  setData,
  setDates,
  setTemps,
  setSelectedDate,
  setSelectedTemp,
  fetchData
} from './actions';

class App extends React.Component {
  fetchData = (evt) => {
    evt.preventDefault();

    var location = encodeURIComponent(this.props.location);

    var urlPrefix = '/cors/http://api.openweathermap.org/data/2.5/forecast?q=';
    var urlSuffix = '&APPID=dbe69e56e7ee5f981d76c3e77bbb45c0&units=metric';
    var url = urlPrefix + location + urlSuffix;

    this.props.dispatch(fetchData(url));

  };

  onPlotClick = (data) => {
    if (data.points) {
      var number = data.points[0].pointNumber;
      this.props.dispatch(setSelectedDate(this.props.dates[number]));
      this.props.dispatch(setSelectedTemp(this.props.temps[number]))
    }
  };

  changeLocation = (evt) => {
    this.props.dispatch(changeLocation(evt.target.value));
  };

  render() {
    var currentTemp = 'not loaded yet';
    if (this.props.data.list) {
      currentTemp = this.props.data.list[0].main.temp;
    }
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>City, Country
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.props.location}
              onChange={this.changeLocation}
            />
          </label>
        </form>
        {/*
          Render the current temperature and the forecast if we have data
          otherwise return null
        */}
        {(this.props.data.list) ? (
          <div>
            {/* Render the current temperature if no specific date is selected */}
            {(this.props.selected.temp) ? (
```

```
            <p>The temperature on { this.props.selected.date } will be { this.props.selecte
          ) : (
            <p>The current temperature is { currentTemp }°C!</p>
          )}
          <h2>Forecast</h2>
          <Plot
            xData={this.props.dates}
            yData={this.props.temps}
            onPlotClick={this.onPlotClick}
            type="scatter"
          />
        </div>
      ) : null}

    </div>
  );
 }
}

// Since we want to have the entire state anyway, we can simply return it as is!
function mapStateToProps(state) {
  return state;
}

export default connect(mapStateToProps)(App);
```

And that's it, everything should be working again now. Look how easy it is to handle our components, how nicely everything is separeted by concern and how easy it would be to add a new feature to our app! That's the power of redux, our application is easier to reason about and to handle, instead of having one massive top-level `App` component we separate the concerns properly.

Now, let's find out how we can make our app so much more performant with immutable datastructures

Additional Material

- Official Redux Docs
- André Staltz' Unidirectional User Interface Architectures
- Egghead.io "Getting started with Redux" Video Course
- Egghead.io Advanced Redux Video Course

Introduction to ImmutableJS

Learn how to enhance your application with immutability to increase the performance and avoid mutation bugs!

As we discovered in Chapter 4: Redux, immutability is quite helpful when developing applications! It makes it so much easier to reason about what is happening to your data, as nothing can be mutated from somewhere entirely different.

The problem is that JavaScript is by default a mutable language. Other developers that don't have this intricate knowledge of immutability might still mess up, mutate the state and break our app in unexpected ways.

Facebook released a second library called `Immutable.js` that adds immutable data structures to JavaScript! Let's see what this looks like.

ImmutableJS exports this nice little `fromJS` function that allows us to create immutable data structures from your standard JavaScript objects and arrays. (it also adds a `toJS` method to make objects and arrays out of them again) Let's create an immutable object:

```
import { fromJS } from 'immutable';

var immutableObject = fromJS({
    some: 'object'
});
```

If you now tried to do `object.some = 'notobject'`, i.e. tried to change the data inside this object, you won't be able to see those changes as those changes would not be applied on the object. That's the power of immutable data structures, you know exactly what they are.

```
import { fromJS } from 'immutable';

let immutableObject = fromJS({
        some: 'object'
```

```
});

immutableObject.some = 'notobject';


console.log(immutableObject); // Doesn't change
```

Now you might be thinking “But then how can we set a property?”. Well, ImmutableJS still let’s us set properties with the `set` and `setIn` methods! Let’s take a look at an example:

```
import { fromJS } from 'immutable';

var immutableObject = fromJS({
        some: 'object'
});

immutableObject.set('some', 'notobject');
console.log(immutableObject);
```

If you run the above code, you’ll still get our initial object again. Why?

Well, since `immutableObject` is immutable, what happens when you `immutableObject.set` is **that a new immutable object is returned with the changes**. No mutation happening, this is kind of like what we did with `Object.assign` for our reducers!

Let’s see if that works:

```
import { fromJS } from 'immutable';

var immutableObject = fromJS({
        some: 'object'
});

newObject = immutableObject.set('some', 'notobject');
console.log(newObject);
```

The changes are there, awesome! `immutableObject` on the other hand still is

our old `{ some: 'object' }` without changes.

As I mentioned before, this is kind of what we did in our redux reducer right? So what would happen if we used ImmutableJS there? Let's try it!

Immutable Redux

In this lesson, we will use Immutable.js in our Weather app that's created using React and Redux

we make the initial state in our reducer an immutable object by using the `fromJS` function! We simply wrap the object that we assign to `initialState` in `fromJS` like so:

```
// reducer.js
/* … */
import { fromJS } from 'immutable';

var initialState = fromJS({
  /* … */
});

/* … */
```

Now we need to rework our reducer. Since our state is now immutable, instead of doing `Object.assign({}, state, { /* … */ })` everywhere we can simply use `state.set`!

Let's showcase this on the `CHANGE_LOCATION` action. This is what our reducer looks like right now:

```
case 'CHANGE_LOCATION':
    return Object.assign({}, state, {
      location: action.location
    });
```

Instead of doing this whole assigning business, we can simply `return state.set('location', action.location)`!

```
case 'CHANGE_LOCATION':
    return state.set('location', action.location);
```

Not only is that a lot cleaner, it's also forcing us to work immutably, which

Not only is that a lot cleaner, it's also forcing us to work immutably, which means we can't accidentally mess something up and introduce weird bugs! 🎉

Let's do the same thing for our `SET_DATA`, `SET_DATES` and `SET_TEMPS` cases:

```
case 'SET_DATA':
  return Object.assign({}, state, {
    data: action.data
  });
case 'SET_DATES':
  return Object.assign({}, state, {
    dates: action.dates
  });
case 'SET_TEMPS':
  return Object.assign({}, state, {
    temps: action.temps
  });
```

This whole block becomes:

```
case 'SET_DATA':
 return state.set('data', fromJS(action.data));
case 'SET_DATES':
 return state.set('dates', fromJS(action.dates));
case 'SET_TEMPS':
 return state.set('temps', fromJS(action.temps));
```

Isn't that nice? Now, here's the last trickery in our reducer, because what do we do for `SET_SELECTED_TEMP` and `SET_SELECTED_DATE`? How do we set `state.selected.temp`?

It turns out Immutable provides us with a really nice function for that called `setIn`. We can use `setIn` to set a nested property by passing in an array of keys we want to iterate through! Let's take a look at that for our `SET_SELECTED_DATE`.

This is what it currently looks like:

```
case 'SET_SELECTED_DATE':
  return Object.assign({}, state, {
    selected: {
      date: action.date,
      temp: state.selected.temp
    }
```

```
  });
```

This works, but you have to agree it's not very nice. With `setIn`, we can simply replace this entire call with this short form:

```
case 'SET_SELECTED_DATE':
  return state.setIn(['selected', 'date'], action.date);
```

So beautiful! Let's do the same thing for `SET_SELECTED_TEMP` and we're done here!

```
case 'SET_SELECTED_TEMP':
  return Object.assign({}, state, {
    selected: {
      date: state.selected.date,
      temp: action.temp
    }
  });
```

becomes

```
case 'SET_SELECTED_TEMP':
  return state.setIn(['selected', 'temp'], action.temp);
```

This is what our reducer looks like finally (let's try running our weather app with our new reducer):

```
import { fromJS } from 'immutable';

var initialState = fromJS({
  location: '',
  data: {},
  dates: [],
  temps: [],
  selected: {
    date: '',
    temp: null
  }
});

export default function mainReducer(state = initialState, action) {
  switch (action.type) {
    case 'CHANGE_LOCATION':
      return state.set('location', action.location);
    case 'SET_DATA':
      return state.set('data', fromJS(action.data));
    case 'SET_DATES':
      return state.set('dates', fromJS(action.dates));
```

```
    case 'SET_TEMPS':
      return state.set('temps', fromJS(action.temps));
    case 'SET_SELECTED_DATE':
      return state.setIn(['selected', 'date'], action.date);
    case 'SET_SELECTED_TEMP':
      return state.setIn(['selected', 'temp'], action.temp);
    default:
      return state;
  }
}
```

If you now try to run your app though, nothing will work and you'll get an error like the following:

`bundle.js:100 mapStateToProps() in Connect(App) must return a plain object. Instead received Map { "location": "", "data": Map {}, "dates": List [], "temps": List [], "selected": Map { "date": "", "temp": null } }.`

This is because in our `App` component we have a `mapStateToProps` function that simply returns the entire state! An easy trick would be to return `state.toJS`, kind of like this:

```
function mapStateToProps(state) {
  return state.toJS();
}
```

Let's try this again (*hint: It will work this time*).

```
import React from 'react';
import './App.css';
import { connect } from 'react-redux';

import Plot from './Plot';
import {
  changeLocation,
  setData,
  setDates,
  setTemps,
  setSelectedDate,
  setSelectedTemp,
  fetchData
} from './actions';

class App extends React.Component {
  fetchData = (evt) => {
    evt.preventDefault();

    var location = encodeURIComponent(this.props.location);

    var urlPrefix = '/cors/http://api.openweathermap.org/data/2.5/forecast?q=';
    var urlSuffix = '&APPID=dbe69e56e7ee5f981d76c3e77bbb45c0&units=metric';
```

```
    var url = urlPrefix + location + urlSuffix;

    this.props.dispatch(fetchData(url));


  };

  onPlotClick = (data) => {
    if (data.points) {
      var number = data.points[0].pointNumber;
      this.props.dispatch(setSelectedDate(this.props.dates[number]));
      this.props.dispatch(setSelectedTemp(this.props.temps[number]))
    }
  };

  changeLocation = (evt) => {
    this.props.dispatch(changeLocation(evt.target.value));
  };

  render() {
    var currentTemp = 'not loaded yet';
    if (this.props.data.list) {
      currentTemp = this.props.data.list[0].main.temp;
    }
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>City, Country
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.props.location}
              onChange={this.changeLocation}
            />
          </label>
        </form>
        {/*
          Render the current temperature and the forecast if we have data
          otherwise return null
        */}
        {(this.props.data.list) ? (
          <div>
            {/* Render the current temperature if no specific date is selected */}
            {(this.props.selected.temp) ? (
              <p>The temperature on { this.props.selected.date } will be { this.props.selecte
            ) : (
              <p>The current temperature is { currentTemp }°C!</p>
            )}
            <h2>Forecast</h2>
            <Plot
              xData={this.props.dates}
              yData={this.props.temps}
              onPlotClick={this.onPlotClick}
              type="scatter"
            />
          </div>
        ) : null}

      </div>
    );
  }
}
```

```
// Since we want to have the entire state anyway, we can simply return it as is!
function mapStateToProps(state) {
  return state.toJS();
}

export default connect(mapStateToProps)(App);
```

Our example above works but there are two downsides to this approach though:

1. Converting from (`fromJS`) and to (`toJS`) JavaScript objects to immutable data structures is *very performance expensive and slow*. This is fine for the `initialState` because we only ever convert that once, but doing that on every render will have an impact on your app.

2. You thus lose the main benefit of ImmutableJS, which is performance!

Now you might be thinking “But if it’s so expensive, how can ImmutableJS have performance as its main benefit?”. To explain that we have to quickly go over how ImmutableJS works.

How ImmutableJS works

In this lesson, we'll take a quick look at the internals of ImmutableJS so that we have a better understanding of how can we apply Immutable.js to our Redux app in a performant manner.

Immutable data structures can't be changed. So when we convert a regular JavaScript object with `fromJS` what ImmutableJS does is loop over every single property and value in the object (including nested objects and arrays) and transfers it to a new, immutable one. (the same thing applies in the other direction for `toJS`)

The problem with standard JavaScript objects is that they have `reference equality`. That means even when two objects have the same content, they're not the same:

```
var object1 = {
        twitter: '@mxstbr'
};

var object2 = {
        twitter: '@mxstbr'
};

console.log(object1 === object2); // -> false
```

In the above example, even though `object1` and `object2` have the exact same contents, they aren't the exact same object and thus aren't equal. To properly check if two variables contain the same thing in JavaScript we'd have to loop over every property and value in those variables (including nested things) and check it against the other object.

That's very, very slow.

Since immutable objects can't ever be changed again, ImmutableJS can *`compute a hash based on the contents of the object`* and store that in a

private field. Since this hash is based on the contents, when Immutable then compares two objects it only has to compare two hashes, i.e. two strings!

That's a lot faster than looping over every property and value and comparing those!

```
import { fromJS } from 'immutable';

var object1 = fromJS({
        twitter: '@mxstbr'
});

var object2 = fromJS({
        twitter: '@mxstbr'
});

console.log(object1.equals(object2)); // -> true 🎉
```

That's nice and all, but how is this helpful in our app?

Utilising ImmutableJS for top performance

Let's roll up our sleeves and apply our learnings from ImmutableJS to our React/Redux weather app.

WE'LL COVER THE FOLLOWING

- `shouldComponentUpdate`
- Additional Material

As a short experiment, try putting a `console.log('RENDER PLOT')` into the `render` method of the `Plot` component:

```
class Plot extends React.Component {
    /* … */
    render() {
        console.log('RENDER PLOT');
    return (
      <div id="plot" ref="plot"></div>
    );
  }
}
```

Now try using the app for a bit, clicking around, request data for different cities. What you might notice is *that the `Plot` rerenders even if we only change the location field and the plot itself stays the exact same*! (Look at your browser console).

```
import React from 'react';
import './App.css';
import { connect } from 'react-redux';

import Plot from './Plot';
import {
  changeLocation,
  setData,
  setDates,
  setTemps,
  setSelectedDate,
  setSelectedTemp,
```

```
  fetchData
} from './actions';

class App extends React.Component {
  fetchData = (evt) => {
    evt.preventDefault();

    var location = encodeURIComponent(this.props.location);

    var urlPrefix = '/cors/http://api.openweathermap.org/data/2.5/forecast?q=';
    var urlSuffix = '&APPID=dbe69e56e7ee5f981d76c3e77bbb45c0&units=metric';
    var url = urlPrefix + location + urlSuffix;

    this.props.dispatch(fetchData(url));

  };

  onPlotClick = (data) => {
    if (data.points) {
      var number = data.points[0].pointNumber;
      this.props.dispatch(setSelectedDate(this.props.dates[number]));
      this.props.dispatch(setSelectedTemp(this.props.temps[number]))
    }
  };

  changeLocation = (evt) => {
    this.props.dispatch(changeLocation(evt.target.value));
  };

  render() {
    var currentTemp = 'not loaded yet';
    if (this.props.data.list) {
      currentTemp = this.props.data.list[0].main.temp;
    }
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>City, Country
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.props.location}
              onChange={this.changeLocation}
            />
          </label>
        </form>
        {/*
          Render the current temperature and the forecast if we have data
          otherwise return null
        */}
        {(this.props.data.list) ? (
          <div>
            {/* Render the current temperature if no specific date is selected */}
            {(this.props.selected.temp) ? (
              <p>The temperature on { this.props.selected.date } will be { this.props.selecte
            ) : (
              <p>The current temperature is { currentTemp }°C!</p>
            )}
            <h2>Forecast</h2>
            <Plot
              xData={this.props.dates}
```

```
              yData={this.props.temps}
              onPlotClick={this.onPlotClick}
              type="scatter"
            />
          </div>
        ) : null}

      </div>
    );
  }
}

// Since we want to have the entire state anyway, we can simply return it as is!
function mapStateToProps(state) {
  return state.toJS();
}

export default connect(mapStateToProps)(App);
```

This is a react feature, react rerenders your entire app whenever something changes. This doesn't necessarily have a massive performance impact on our current application, but it'll definitely bite you in a production application! So, what can we do against that?

`shouldComponentUpdate`

React provides us with a nice lifecycle method called `shouldComponentUpdate` which allows us to regulate when our components should rerender. As an example, try putting this into your `Plot`:

```
class Plot extends React.Component {
    shouldComponentUpdate(nextProps) {
        return false;
    }
    /* … */
}
```

Now try loading some data and rendering a plot. What you see is that *the plot never renders*. This is because we're basically telling react above that no matter what data comes into our component, it should never render the `Plot`! On the other hand, if we `return true` from there we'd have the default behaviour back, i.e. rerender whenever new data comes in.

```
import React from 'react';

class Plot extends React.Component {
  shouldComponentUpdate(nextProps) {
                return false;
```

```
      }

  drawPlot = () => {
    Plotly.newPlot('plot', [{
      x: this.props.xData,
      y: this.props.yData,
      type: this.props.type
    }], {
      margin: {
        t: 0, r: 0, l: 30
      },
      xaxis: {
        gridcolor: 'transparent'
      }
    }, {
      displayModeBar: false
    });

    document.getElementById('plot').on(
      'plotly_click', this.props.onPlotClick);
  }

  componentDidMount() {
    this.drawPlot();
  }

  componentDidUpdate() {
    this.drawPlot();
  }

  render() {
    console.log('RENDER PLOT');
    return (
      <div id="plot"></div>
    );
  }
}


export default Plot;
```

As I've hinted with the variable above, `shouldComponentUpdate` gets passed `nextProps`. This means, in theory, we could check if the props of the `Plot` have changed and only rerender if that happens, right? Something like this:

```
class Plot extends React.Component {
    shouldComponentUpdate(nextProps) {
        return this.props !== nextProps;
    }
    /* … */
}
```

Well, here we hit the problem we talked about above. (`{ twitter: '@mxstbr' } !== { twitter: '@mxstbr' }`) Those will always be different since they might

have the same content, but they won't be the same object!

This is where ImmutableJS comes in, because while we could do a *deep comparison* of those two objects, it's a lot cheaper if we could just do this:

```
class Plot extends React.Component {
    shouldComponentUpdate(nextProps) {
        return !this.props.equals(nextProps);
    }
    /* … */
}
```

Let's try getting some immutable data to our `Plot`!

In our `mapStateToProps` function, instead of returning `state.toJS()` we should just return the immutable state. The problem is that redux expects the value we return from `mapStateToProps` to be a standard javascript object, and it'll throw an error if we just do `return state;` and nothing will work.

So let's return an object from `mapStateToProps` that has a `redux` field instead:

```
function mapStateToProps(state) {
  return {
        redux: state
    };
}
```

Then, in our `App` we now have access to `this.props.redux`! We can access properties in there with `this.props.redux.get` (and `getIn`), so let's replace all instances where we access the state with that.

Let's start from the top, in `fetchData`. There's only a single `this.props.location` in there, which we replace with `this.props.redux.get('location')`:

```
class App extends React.Component {
  fetchData = (evt) => {
    evt.preventDefault();

    var location = encodeURIComponent(this.props.redux.get('location'));

    /* … */
```

```
  };

  onPlotClick = (data) => {/* … */};

  changeLocation = (evt) => {/* … */};

  render() {/* … */}
}
```

We don't access the props at all in `onPlotClick` and `changeLocation`, so we can skip those!

In `render`, the first access is already a bit more difficult – we want to replace `this.props.data.list`… Do you remember how to do that?

…

With `getIn`! Like this:

```
class App extends React.Component {
    fetchData = (evt) => {/* … */};

    onPlotClick = (data) => {/* … */};

    changeLocation = (evt) => {/* … */};

    render() {
        var currentTemp = 'not loaded yet';
        if (this.props.redux.getIn(['data', 'list'])) {
            /* … */
        }
        return (/* … */);
    }
}
```

Now, for the next one (`this.props.data.list[0].main.temp`) you might think of writing `this.props.redux.getIn(['data', 'list'])[0].main.temp`, but the problem is that `this.props.redux.getIn(['data', 'list'])` is an immutable array too!

So, instead we can just further use `getIn`:

```
class App extends React.Component {
  fetchData = (evt) => {/* … */};
```

```
  onPlotClick = (data) => {/* … */};

  changeLocation = (evt) => {/* … */};

  render() {
    var currentTemp = 'not loaded yet';
    if (this.props.redux.getIn(['data', 'list'])) {
      currentTemp = this.props.redux.getIn(['data', 'list', '0', 'mai
n', 'temp']);
    }
    return (/* … */);
  }
}
```

Similarly, we are going to fix `onPlotClick`:

```
onPlotClick = (data) => {
    if (data.points) {
      var number = data.points[0].pointNumber;
      this.props.dispatch(setSelectedDate(this.props.redux.getIn(['date
s', number])));
      this.props.dispatch(setSelectedTemp(this.props.redux.getIn(['temp
s', number])))
    }
};
```

Now let's put all of this in action (*It will still not work but you should try running it. Can you guess what's the issue?*)

```
import React from 'react';
import './App.css';
import { connect } from 'react-redux';
import { fromJS } from 'immutable';

import Plot from './Plot';
import {
  changeLocation,
  setData,
  setDates,
  setTemps,
  setSelectedDate,
  setSelectedTemp,
  fetchData
} from './actions';

class App extends React.Component {
  fetchData = (evt) => {
    evt.preventDefault();
```

```
    var location = encodeURIComponent(this.props.redux.get('location'));

    var urlPrefix = '/cors/http://api.openweathermap.org/data/2.5/forecast?q=';

    var urlSuffix = '&APPID=dbe69e56e7ee5f981d76c3e77bbb45c0&units=metric';
    var url = urlPrefix + location + urlSuffix;

    this.props.dispatch(fetchData(url));

  };

  onPlotClick = (data) => {
    if (data.points) {
      var number = data.points[0].pointNumber;
      this.props.dispatch(setSelectedDate(this.props.redux.getIn(['dates', number])));
      this.props.dispatch(setSelectedTemp(this.props.redux.getIn(['temps', number])))
    }
  };

  changeLocation = (evt) => {
    this.props.dispatch(changeLocation(evt.target.value));
  };

  render() {
    var currentTemp = 'not loaded yet';
    if (this.props.redux.getIn(['data', 'list'])) {
      currentTemp = this.props.redux.getIn(['data', 'list', '0', 'main', 'temp']);
    }
    return (
      <div>
        <h1>Weather</h1>
        <form onSubmit={this.fetchData}>
          <label>I want to know the weather for
            <input
              placeholder={"City, Country"}
              type="text"
              value={this.props.redux.get('location')}
              onChange={this.changeLocation}
            />
          </label>
        </form>
        {/*
          Render the current temperature and the forecast if we have data
          otherwise return null
        */}
        {(this.props.redux.getIn(['data', 'list'])) ? (
          <div>
            {/* Render the current temperature if no specific date is selected */}
            { this.props.redux.getIn(['selected', 'temp']) ? (
              <p>The temperature on { this.props.redux.getIn(['selected', 'date'])} will be {
            ) : (
              <p>The current temperature is { currentTemp }°C!</p>
            )}
            <h2>Forecast</h2>
            <Plot
              xData={this.props.redux.get('dates')}
              yData={this.props.redux.get('temps')}
              onPlotClick={this.onPlotClick}
              type="scatter"
            />
          </div>
        ) : null}
```

```
      </div>
    );
  }
}

// Since we want to have the entire state anyway, we can simply return it as is!
function mapStateToProps(state) {
  return {
                redux: state
        };
}

export default connect(mapStateToProps)(App);
```

As you might've noticed, this doesn't work though, the Plot doesn't render. Why? Well, take a look at how we pass in the data:

```
<Plot
    xData={this.props.redux.get('dates')}
    yData={this.props.redux.get('temps')}
    onPlotClick={this.onPlotClick}
    type="scatter"
/>
```

Let's take a peek at the only method where we use `this.props.xData` and `this.props.yData` in our `Plot` component:

```
// Plot.js

class Plot extends React.Component {
  drawPlot = () => {
    Plotly.newPlot('plot', [{
      x: this.props.xData,
      y: this.props.yData,
      type: this.props.type
    }], {/* … */}, {/* … */});
    /* … */
  }

  componentDidMount() {/* … */}
  componentDidUpdate() {/* … */}
  render() {/* … */}
}
```

This is where `toJS` comes in! Let's do this:

```
// Plot.js
```

```
class Plot extends React.Component {
  drawPlot = () => {
    Plotly.newPlot('plot', [{
      x: this.props.xData.toJS(),
      y: this.props.yData.toJS(),
      type: this.props.type
    }], {/* … */}, {/* … */});
    /* … */
  }

  componentDidMount() {/* … */}
  componentDidUpdate() {/* … */}
  render() {/* … */}
}
```

And now everything works again!

```
import React from 'react';

class Plot extends React.Component {
  drawPlot = () => {
    Plotly.newPlot('plot', [{
      x: this.props.xData.toJS(),
      y: this.props.yData.toJS(),
      type: this.props.type
    }], {
      margin: {
        t: 0, r: 0, l: 30
      },
      xaxis: {
        gridcolor: 'transparent'
      }
    }, {
      displayModeBar: false
    });

    document.getElementById('plot').on(
      'plotly_click', this.props.onPlotClick);
  }

  componentDidMount() {
    this.drawPlot();
  }

  componentDidUpdate() {
    this.drawPlot();
  }

  render() {
    console.log('RENDER PLOT');
    return (
      <div id="plot"></div>
    );
```

```
  }
}



export default Plot;
```

It sort of works but are you looking at your console. Remember those frequent `RENDER PLOT` statements in your console. They are still there (*yes we haven't removed them as yet*) and they are getting printed with every click on the graph even when there's no new data to update the graph.

Hence, we still haven't solved the original problem though, the `Plot` still rerenders everytime something changes, even if it's not related to the Plot. Really, *the only time we ever want that component to rerender is when either* `xData` *or* `yData` *changes!*

Let's apply our knowledge of ImmutableJS and of `shouldComponentUpdate`, and fix this together. Let's check if `this.props.xData` and `this.props.yData` are the same and only rerender if one of them changed:

```
import React from 'react';

class Plot extends React.Component {
  shouldComponentUpdate(nextProps) {
                const xDataChanged = !this.props.xData.equals(nextProps.xData);
                const yDataChanged = !this.props.yData.equals(nextProps.yData);

                return xDataChanged || yDataChanged;
        }

  drawPlot = () => {
    Plotly.newPlot('plot', [{
      x: this.props.xData.toJS(),
      y: this.props.yData.toJS(),
      type: this.props.type
    }], {
      margin: {
        t: 0, r: 0, l: 30
      },
      xaxis: {
        gridcolor: 'transparent'
      }
    }, {
      displayModeBar: false
    });

    document.getElementById('plot').on(
      'plotly_click', this.props.onPlotClick);
  }

  componentDidMount() {
    this.drawPlot();
```

```
  }

  componentDidUpdate() {

    this.drawPlot();
  }

  render() {
    console.log('RENDER PLOT');
    return (
      <div id="plot"></div>
    );
  }
}


export default Plot;
```

Now try clicking around and loading different cities.

The `Plot` component now only rerenders when new data comes in! 🎉

We've entirely gotten rid of the continuous rerenders, and only rerender when it's *really* necessary! This is awesome!

Let's explore how we can make sure our app works the way we expect it to, no matter who's working on it i.e. lets test it.

Additional Material

- Official ImmutableJS docs
- Introduction to Immutable.js and Functional Programming Concepts
- Pros and Cons of using immutability with React.js

Running our Weather app on your machine

We'll look at how to run our app on your machine so that you can run tests against the app

WE'LL COVER THE FOLLOWING

- Prerequisites
- First steps
- Install npm modules

Prerequisites

Node.js is a JavaScript runtime for your terminal. Don't worry too much about it, but it's used by some tools we'll be using to build our application. If you don't have it installed already (check by running `node -v` in your terminal, which should print a version number) head over to nodejs.org, follow the instructions there to install the latest version (v6 at the time of this writing) and you're good to go!

First steps

Facebook recently open sourced a neat little tool called `create-react-app` that allows us to very easily get started with our React app! It includes all the necessary build tools and transpilation steps to just get stuff done.

Let's install it with `npm`:

```
npm install -g create-react-app
```

As soon as that's finished you now have access to the `create-react-app` command in your terminal! Let's create our barebones weather app:

```
create-react-app weather-app
```

> The argument to `create-react-app`, in our case `weather-app`, tells the utility what to name the folder it'll create. Since we're creating a weather app, `weather-app` seems like a solid choice!

Install npm modules

During the lessons, you might have noticed that we have used a few npm modules. Let's install those modules in your app directory

```
npm install --save redux react-redux
npm install --save redux-thunk
npm install --save immutable
```

Now comes the hardest part (*just kidding* :)).

Copy the code from the files below and put it into the respective files. `index.js` is already there and you should create the other files e.g. `App.js`, `Plot.js`, `reducers.js`, `actions.js` etc.

```
import React from 'react';

class Plot extends React.Component {
  shouldComponentUpdate(nextProps) {
                const xDataChanged = !this.props.xData.equals(nextProps.xData);
                const yDataChanged = !this.props.yData.equals(nextProps.yData);

                return xDataChanged || yDataChanged;
        }

  drawPlot = () => {
    Plotly.newPlot('plot', [{
      x: this.props.xData.toJS(),
      y: this.props.yData.toJS(),
      type: this.props.type
    }], {
      margin: {
        t: 0, r: 0, l: 30
      },
      xaxis: {
        gridcolor: 'transparent'
      }
    }, {
      displayModeBar: false
    });

    document.getElementById('plot').on(
      'plotly_click', this.props.onPlotClick);
```

```
      plotly_click', this.props.onPlotClick);
  }


  componentDidMount() {
    this.drawPlot();
  }

  componentDidUpdate() {
    this.drawPlot();
  }

  render() {
    console.log('RENDER PLOT');
    return (
      <div id="plot"></div>
    );
  }
}


export default Plot;
```

Thankfully, create-react-app includes a simple server so instead of having to open the index.html file manually we can simply run `npm run start` in the weather-app directory and see our application at localhost:3000!

Now that we have our app running on your machine, lets jump into testing it.

Testing your app with Jest

Learn how to test your React/Redux app thoroughly, including actions, reducers and components!

WE'LL COVER THE FOLLOWING

- Unit testing
  - Basics
  - Jest
  - Redux

Testing apps is something that a lot of developers *should* be doing, but a lot of them don't. It has a bunch of really nice benefits:

1. **You catch bugs before they happen.** The obvious out of the bunch, but nontheless very important. We all make mistakes, and with an automated test suite (an array of tests) we make sure no users has to ever see those!
2. **Tests are executable documentation.** This is in my opinion the biggest benefit. With a function that is well tested you can immediately figure out what it's used for, how you should use it and what to expect it to do! No more tedious documentation writing.
3. **Tests save time.** This might be a bit counterintuitive, since writing tests takes away time you'd spend writing your app. But then you still have to do Q&A, manual testing to make sure you didn't break something else with a change! Automated tests save so much time that would be spent manually doing Q&A and finding bugs!
4. **You will write better code.** Some code is harder to test, some easier. You'll start writing easier testable code, which automatically is better code!

With this in mind, there's no way we could not test our application! Let's get started!

Unit testing

Unit testing is the practice of testing the smallest possible *units* of our code. In JavaScript, those are functions. We run our tests and automatically verify that our functions do the thing we expect them to do. We assert that, given a set of inputs, our functions return the proper values and handle problems.

We'll be using the Jest test framework by facebook. It was written to help test react apps, and is perfect for that purpose! It makes writing tests as easy as speaking - you `describe` a unit of your code and `expect` `it` to do the correct thing.

Thankfully, `create-react-app` comes with it installed by default so we won't need to do any setup! Simply enter `npm run test` into your terminal to run the tests we'll write below.

> Note: **You need version 0.3.0 or higher of `create-react-app` for Jest support.** To check which version you have installed look into the `package.json` of the `weather-app` folder. If it's lower than `0.3.0` follow the upgrade guide to get the testing setup.

Basics

For the sake of this guide, lets pretend we're testing this function. It's situated in the `src/add.js` file:

```
// src/add.js

export function add(x, y) {
  return x + y;
}
```

Jest

Jest is our unit testing framework. Its API, which we write tests with, is speech like and easy to use.

> Note: This is the official documentation of Jest.

We're going to add a second file called `add.test.js` in a subfolder called `src/__tests__/` with our unit tests inside. Running said unit tests requires us to enter `npm run test -- src/__tests__/add.test.js` into the command line.

First, we `import` the function in our `add.test.js` file:

```
// src/__tests__/add.test.js

import { add } from '../add.js';
```

Second, we `describe` our function:

```
describe('add()', function() {

});
```

Third, we tell Jest what `it` (our function) should do:

```
describe('add()', function() {
  it('adds two numbers', function() {

  });

  it('doesnt add the third number', function() {

  });
});
```

Now we have to `expect` our little function to return the same thing every time given the same input. We're going to test that our little function correctly adds two numbers first. We are going to take some chosen inputs, and `expect` the result `toEqual` the corresponding output:

```
// [...]
it('adds two numbers', function() {
  expect(add(2, 3)).toEqual(5);
});
// [...]
```

Lets add the second test, which determines that our function doesn't add the

third number if one is present:

```
// [...]
it('doesnt add the third number', function() {
 expect(add(2, 3, 5)).toEqual(add(2, 3));
});
// [...]
```

> Note: Notice that we call `add` in `toEqual`. I won't tell you why, but just think about what would happen if we rewrote the expect as `expect(add(2, 3, 5)).toEqual(5)` and somebody broke something in the add function. What would this test actually... test?

Should our function work, Jest will show this output when running the tests:

```
PASS  src/__tests__/add.test.js (0.537s)
2 tests passed (2 total in 1 test suite, run time 0.557s)
```

Lets say an unnamed colleague of ours breaks our function:

```
// add.js

export function add(x, y) {
  return x * y;
}
```

Oh no, now our function doesn't add the numbers anymore, it multiplies them! Imagine the consequences to our code that uses the function!

Thankfully, we have unit tests in place. Because we run the unit tests before we deploy our application, we see this output:

```
 FAIL  src/__tests__/add.test.js (0.535s)
● add() › it adds two numbers
  - Expected 6 to equal 5.
        at Object.<anonymous> (__tests__/add.test.js:5:65)
1 test failed, 1 test passed (2 total in 1 test suite, run time 0.564s)
```

This tells us that something is broken in the add function before any users get the code! Congratulations, you just saved time and money!

Redux

The nice thing about Redux is that it makes our data flow entirely consist of “pure” functions. Pure functions are functions that return the same output with the same input everytime – they don’t have any side effects!

Let’s test our actions first!

Testing Actions

We'll test Redux actions in our Weather app that's built with React and Redux.

Create a new file called `actions.test.js` in the `src/__tests__ /` folder. (create that if you haven't already) Let's start by testing the good ol' `changeLocation` action. Add the default structure, we'll need to import the action we want to test and `describe` "actions" and `changeLocation`:

```
// actions.test.js
import {
  changeLocation
} from '../actions';

describe('actions', function() {
  describe('changeLocation', function () {

  });
});
```

There's two things we want to verify of our action function: that it has the correct type and that it passes on the data we tell it to pass on. Let's verify the type of the `changeLocation` action is `'CHANGE_LOCATION'`:

```
// actions.test.js
/* … */
describe('changeLocation', function () {
  it('should have a type of "CHANGE_LOCATION"', function() {
    expect(changeLocation().type).toEqual('CHANGE_LOCATION');
  });
});
```

Run `npm run test` in the console and this is what you should see:

```
PASS  src/__tests__/actions.test.js (0.525s)
1 test passed (1 total in 1 test suite, run time 0.55s)
```

Nice, let's verify that it passes on the location we pass into it:

```
// actions.test.js
/* … */
describe('changeLocation', function () {
  it('should have a type of "CHANGE_LOCATION"', function() {
    expect(changeLocation().type).toEqual('CHANGE_LOCATION');
  });

  it('should pass on the location we pass in', function() {
    var location = 'Vienna, Austria';
    expect(changeLocation(location).location).toEqual(location);
  });
});
```

Nice! Now let's do the same thing for the `setSelectedDate` and `setSelectedTemp` action! First, `import` those two actions at the of the file and add the `describe` and `it`s:

```
describe('actions', function() {
  describe('changeLocation', function () { /* … */ });

  describe('setSelectedDate', function() {
    it('should have a type of SET_SELECTED_DATE', function() { });

        it('should pass on the date we pass in', function() { });
  });

  describe('setSelectedTemp', function() {
    it('should have a type of SET_SELECTED_TEMP', function() { });

        it('should pass on the temp we pass in', function() { });
  });
});
```

First let's verify our `setSelectedDate` works as expected:

```
describe('actions', function() {
  describe('changeLocation', function () { /* … */ });

  describe('setSelectedDate', function() {
    it('should have a type of SET_SELECTED_DATE', function() {
            expect(setSelectedDate().type).toEqual('SET_SELECTED_DATE');
        });
```

```
        it('should pass on the date we pass in', function() {
            var date = '2016-01-01';
            expect(setSelectedDate(date).date).toEqual(date);
        });
  });

  describe('setSelectedTemp', function() {
    it('should have a type of SET_SELECTED_TEMP', function() { });

        it('should pass on the temp we pass in', function() { });
  });
});
```

and then our `setSelectedTemp`:

```
describe('actions', function() {
  describe('changeLocation', function () { /* … */ });

  describe('setSelectedDate', function() {
    it('should have a type of SET_SELECTED_DATE', function() {
            expect(setSelectedDate().type).toEqual('SET_SELECTED_DATE');
        });

        it('should pass on the date we pass in', function() {
            var date = '2016-01-01';
            expect(setSelectedDate(date).date).toEqual(date);
        });
  });

  describe('setSelectedTemp', function() {
    it('should have a type of SET_SELECTED_TEMP', function() {
            expect(setSelectedTemp().type).toEqual('SET_SELECTED_TEMP');
        });

        it('should pass on the temp we pass in', function() {
            var temp = '31';
            expect(setSelectedTemp(temp).temp).toEqual(temp);
        });
  });
});
```

Not too hard, huh? Run `npm run test` in your console now, and this is what you should see:

```
PASS  src/__tests__/actions.test.js (0.531s)
6 tests passed (6 total in 1 test suite, run time 0.554s)
```

Now go on and test the other actions too, I'll be here waiting for you! (skip the `fetchData` action, one negative aspect of thunks is how hard they are to test so we'll skip it)

Back? Everything tested? You should now see something like this in your console when running `npm run test`:

```
PASS  src/__tests__/actions.test.js (0.357s)
12 tests passed (12 total in 1 test suite, run time 0.384s)
```

This isn't the nicest output though, if you run `npm run test -- --verbose` you should see a much nicer list of tests that passed like so:

```
PASS  src/__tests__/actions.test.js (0.364s)
 actions
   changeLocation
     ✓ it should have a type of CHANGE_LOCATION (5ms)
     ✓ it should pass on the location we pass in (1ms)
   setSelectedDate
     ✓ it should have a type of SET_SELECTED_DATE (1ms)
     ✓ it should pass on the date we pass in
   setSelectedTemp
     ✓ it should have a type of SET_SELECTED_TEMP (1ms)
     ✓ it should pass on the temp we pass in
   setData
     ✓ it should have a type of SET_DATA
     ✓ it should pass on the data we pass in (1ms)
   setDates
     ✓ it should have a type of SET_DATES
     ✓ it should pass on the dates we pass in
   setTemps
     ✓ it should have a type of SET_TEMPS (1ms)
     ✓ it should pass on the temps we pass in

12 tests passed (12 total in 1 test suite, run time 0.392s)
```

And this is what your `actions.test.js` file could look like:

```
// actions.test.js
```

```
import {
    changeLocation,
    setSelectedDate,
    setSelectedTemp,
    setData,
    setDates,
    setTemps
} from '../actions';

describe('actions', function() {
    describe('changeLocation', function() {
        it('should have a type of CHANGE_LOCATION', function() {
            expect(changeLocation().type).toEqual('CHANGE_LOCATION');
        });

        it('should pass on the location we pass in', function() {
            var location = 'Vienna, Austria';
            expect(changeLocation(location).location).toEqual(location);
        });
    });

    describe('setSelectedDate', function() {
        it('should have a type of SET_SELECTED_DATE', function() {
            expect(setSelectedDate().type).toEqual('SET_SELECTED_DATE');
        });

        it('should pass on the date we pass in', function() {
            var date = '2016-01-01';
            expect(setSelectedDate(date).date).toEqual(date);
        });
    });

    describe('setSelectedTemp', function() {
        it('should have a type of SET_SELECTED_TEMP', function() {
            expect(setSelectedTemp().type).toEqual('SET_SELECTED_TEMP');
        });

        it('should pass on the temp we pass in', function() {
            var temp = '31';
            expect(setSelectedTemp(temp).temp).toEqual(temp);
        });
    });

    describe('setData', function() {
        it('should have a type of SET_DATA', function() {
            expect(setData().type).toEqual('SET_DATA');
```

```
        });


        it('should pass on the data we pass in', function() {
            var data = { some: 'data' };
            expect(setData(data).data).toEqual(data);
        });
    });

    describe('setDates', function() {
        it('should have a type of SET_DATES', function() {
            expect(setDates().type).toEqual('SET_DATES');
        });

        it('should pass on the dates we pass in', function() {
            var dates = ['2016-01-01', '2016-01-02'];
            expect(setDates(dates).dates).toEqual(dates);
        });
    });

    describe('setTemps', function() {
        it('should have a type of SET_TEMPS', function() {
            expect(setTemps().type).toEqual('SET_TEMPS');
        });

        it('should pass on the temps we pass in', function() {
            var temps = ['31', '32'];
            expect(setTemps(temps).temps).toEqual(temps);
        });
    });
});
```

Perfect, that part of our app is now comprehensively tested and we'll know as soon as somebody breaks something! Onwards to the reducer!

Testing Reducers

We'll test Redux reducers in our Weather app that's built with React and Redux.

The reducer is, again, a pure function! It's quite easy to see what we need to validate actually, basically every `case` of our `switch` needs to have a test:

```
export default function mainReducer(state = initialState, action) {
  switch (action.type) {
    case 'CHANGE_LOCATION':
      return state.set('location', action.location);
    case 'SET_DATA':
      return state.set('data', fromJS(action.data));
    case 'SET_DATES':
      return state.set('dates', fromJS(action.dates));
    case 'SET_TEMPS':
      return state.set('temps', fromJS(action.temps));
    case 'SET_SELECTED_DATE':
      return state.setIn(['selected', 'date'], action.date);
    case 'SET_SELECTED_TEMP':
      return state.setIn(['selected', 'temp'], action.temp);
    default:
      return state;
  }
}
```

Let's showcase this on the `'CHANGE_LOCATION'` case, first create a `reducer.test.js` file in the `__tests__ /` directory, import the reducer and add the basic structure:

```
// __tests__/reducer.test.js
import mainReducer from '../reducer';

describe('mainReducer', function() {

});
```

The first branch of the switch statement we'll test is the `default` one – if we

don't pass any state and an empty action in it should return the initial state.

The thing is that the `initialState` is an immutable object, so we'll need to import `fromJS` too:

```
// __tests__/reducer.test.js
import mainReducer from '../reducer';
import { fromJS } from 'immutable';

describe('mainReducer', function() {
  it('should return the initial state', function() {
    expect(mainReducer(undefined, {})).toEqual(fromJS({
            location: '',
          data: {},
          dates: [],
          temps: [],
          selected: {
            date: '',
            temp: null
          }
        }));
  });
});
```

You should now see this output:

```
PASS  src/__tests__/actions.test.js (0.365s)
PASS  src/__tests__/reducer.test.js (0.215s)
13 tests passed (13 total in 2 test suites, run time 0.519s)
```

Brilliant! Let's showcase how we can test specific actions, again using our beloved `'CHANGE_LOCATION'` one.

First, add a new `it` explaining what the reducer should do:

```
// __tests__/reducer.test.js
import mainReducer from '../reducer';
import { fromJS } from 'immutable';

describe('mainReducer', function() {
  it('should return the initial state', function() {/* … */});

  it('should react to an action with the type CHANGE_LOCATION', function
() {
```

```
  });
});
```

Then, validate that the reducer changes the `location` field in the state correctly:

```
it('should react to an action with the type CHANGE_LOCATION', function() {
  var location = 'Vienna, Austria';
  expect(mainReducer(undefined, {
    type: 'CHANGE_LOCATION',
    location: location
  })).toEqual(fromJS({
    location: location,
    data: {},
    dates: [],
    temps: [],
    selected: {
      date: '',
      temp: null
    }
  }));
});
```

Now we know that our action returns an object with a `type` of `"CHANGE_LOCATION"` and that our reducer changes the `location` field in the state correctly in response to an object with a `type` of `"CHANGE_LOCATION"`! Brilliant!

Let's do the same thing for our `'SET_DATES'` case, first add the `it`:

```
// __tests__/reducer.test.js
import mainReducer from '../reducer';
import { fromJS } from 'immutable';

describe('mainReducer', function() {
  it('should return the initial state', function() {/* … */});

  it('should react to an action with the type CHANGE_LOCATION', function
() {/* … */});

  it('should react to an action with the type SET_DATES', function() {
```

```
  });
});
```

Then make sure our reducer acts accordingly:

```
it('should react to an action with the type SET_DATES', function() {
  var dates = ['2016-01-01', '2016-02-02'];
  expect(mainReducer(undefined, {
    type: 'SET_DATES',
    dates: dates
  })).toEqual(fromJS({
    location: '',
    data: {},
    dates: dates,
    temps: [],
    selected: {
      date: '',
      temp: null
    }
  }));
});
```

Not too hard, eh? That's the power of redux!

Now that we have showcased how it works with those two examples, go ahead and test the other cases too!

Done? This is what your terminal output should look like when running `npm run test -- --verbose`:

```
PASS  src/__tests__/actions.test.js (0.365s)
 actions
   changeLocation
     ✓ it should have a type of CHANGE_LOCATION (5ms)
     ✓ it should pass on the location we pass in (1ms)
   setSelectedDate
     ✓ it should have a type of SET_SELECTED_DATE (1ms)
     ✓ it should pass on the date we pass in
   setSelectedTemp
     ✓ it should have a type of SET_SELECTED_TEMP (1ms)
     ✓ it should pass on the temp we pass in
   setData
     ✓ it should have a type of SET_DATA (1ms)
     ✓ it should pass on the data we pass in
   setDates
```

```
    ✓ it should have a type of SET_DATES
    ✓ it should pass on the dates we pass in (1ms)

  setTemps
    ✓ it should have a type of SET_TEMPS
    ✓ it should pass on the temps we pass in (1ms)

PASS  src/__tests__/reducer.test.js (0.515s)
 mainReducer
   ✓ it should return the initial state (7ms)
   ✓ it should react to an action with the type 'CHANGE_LOCATION' (1ms)
   ✓ it should react to an action with the type 'SET_DATA' (3ms)
   ✓ it should react to an action with the type 'SET_DATES' (4ms)
   ✓ it should react to an action with the type 'SET_TEMPS' (2ms)
   ✓ it should react to an action with the type 'SET_SELECTED_DATE' (1ms)
   ✓ it should react to an action with the type 'SET_SELECTED_TEMP'

19 tests passed (19 total in 2 test suites, run time 0.819s)
```

If you do not have all the 7 cases in your reducer tested, go back and try to do them all! It'll strengthen your testing muscle and help you get used to thinking this way!

When your output looks like the output above, you're done! This is what your `reducer.test.js` file should look like:

```
import mainReducer from '../reducer';
import { fromJS } from 'immutable';

describe('mainReducer', function() {
    it('should return the initial state', function() {
        expect(mainReducer(undefined, {})).toEqual(fromJS({
            location: '',
          data: {},
          dates: [],
          temps: [],
          selected: {
            date: '',
            temp: null
          }
        }));
    });

    it("should react to an action with the type 'CHANGE_LOCATION'", functi
on() {
        var location = 'Vienna, Austria';
```

```
    expect(mainReducer(undefined, {
        type: 'CHANGE_LOCATION',
        location: location
    })).toEqual(fromJS({
        location: location,
      data: {},
      dates: [],
      temps: [],
      selected: {
        date: '',
        temp: null
      }
    }));
});

it("should react to an action with the type 'SET_DATA'", function() {
    var data = { some: 'data' };
    expect(mainReducer(undefined, {
        type: 'SET_DATA',
        data: data
    })).toEqual(fromJS({
        location: '',
      data: data,
      dates: [],
      temps: [],
      selected: {
        date: '',
        temp: null
      }
    }));
});

it("should react to an action with the type 'SET_DATES'", function() {
    var dates = ['2016-01-01', '2016-02-02'];
    expect(mainReducer(undefined, {
        type: 'SET_DATES',
        dates: dates
    })).toEqual(fromJS({
        location: '',
      data: {},
      dates: dates,
      temps: [],
      selected: {
        date: '',
        temp: null
      }
```

```
        }));
    });


    it("should react to an action with the type 'SET_TEMPS'", function() {
        var temps = ['31', '32'];
        expect(mainReducer(undefined, {
            type: 'SET_TEMPS',
            temps: temps
        })).toEqual(fromJS({
            location: '',
          data: {},
          dates: [],
          temps: temps,
          selected: {
            date: '',
            temp: null
          }
        }));
    });

    it("should react to an action with the type 'SET_SELECTED_DATE'", func
tion() {
        var date = '2016-02-01'
        expect(mainReducer(undefined, {
            type: 'SET_SELECTED_DATE',
            date: date
        })).toEqual(fromJS({
            location: '',
          data: {},
          dates: [],
          temps: [],
          selected: {
            date: date,
            temp: null
          }
        }));
    });

    it("should react to an action with the type 'SET_SELECTED_TEMP'", func
tion() {
        var temp = '31';
        expect(mainReducer(undefined, {
            type: 'SET_SELECTED_TEMP',
            temp: temp
        })).toEqual(fromJS({
            location: '',
```

```
          data: {},
          dates: [],

          temps: [],
          selected: {
            date: '',
            temp: temp
          }
        }));
    });
});
```

Onwards to testing our components!

Testing Components

We'll test components in our Weather app that's built with React and Redux.

WE'LL COVER THE FOLLOWING

- Setup

When testing our components, the one thing we want to verify is that they render the same output as last time the test was ran. Since they are bound to change very often, more specific tests are more of a burden than a help. If the test fails, but we've manually verified the new output is correct we should be able to quickly tell that to our testing framework without much effort.

Exactly for that purpose, Jest recently added support for **component snapshots**. Component snapshots are generated on the first test run and saved in files in your project. They should be checked into version control if you have one, and code reviews should include them.

By having those snapshots after the first test run, we can immediately verify if our component output was changed. If any change happened and we manually verified the new version is correct we can run `jest -u` to update the existing snapshots!

Setup

To render the components without opening a browser we'll have to install the `react-test-renderer`. It allows us to render the component to a JSON object!

```
npm install --save-dev react-test-renderer
```

Let's create a new file and add the basic testing code. We'll be starting with the App component, so `import` that for now:

```
//   tests  /components.test.js
```

```
// __tests__/components.test.js

import React from 'react';

import renderer from 'react-test-renderer';
import App from '../App';

describe('components', function() {
  describe('<App />', function() {

  });
});
```

The thing we want to verify in our `App` component is that it renders without throwing an error and taking a snapshot so we know when the output changes. Let's add an `it` to that effect:

```
// __tests__/components.test.js

import React from 'react';
import renderer from 'react-test-renderer';
import App from '../App';

describe('components', function() {
  describe('<App />', function() {
    it('renders correctly', function() {

    });
  });
});
```

Let's now create a renderer, render our `<App />` component to JSON and expect that to match the snapshot:

```
// __tests__/components.test.js

import React from 'react';
import renderer from 'react-test-renderer';
import App from '../App';

describe('components', function() {
  describe('<App />', function() {
    it('renders correctly', function() {
      var tree = renderer.create(<App />).toJSON();
      expect(tree).toMatchSnapshot();
    });
```

```
  });
});
```

Try running this though, and you'll get this error:

```
- Invariant Violation: Could not find "store" in either the context or pro
ps of "Connect(App)". Either wrap the root component in a <Provider>, or e
xplicitly pass "store" as a prop to "Connect(App)".
```

Ugh, what's this now? Couldn't find `store`? What?

Remember what we export from the `App.js` file? The `react-redux` `connect`ed component! What we want to test though is the actual component itself, so we'll have to export that too:

```
// App.js

/* … */

export class App extends React.Component {/* … */}

/* … */

export default connect(mapStateToProps)(App);
```

Awesome! Now we need to change the `import` in our test file to reference that new export and everything should work, right?

```
// __tests__/components.test.js

import { App } from '../App';

/* … */
```

Well, no, but we're getting a different error now! That's a good sign!

```
- TypeError: Cannot read property 'getIn' of undefined
```

Remember what `getIn` is used for? ImmutableJS! If you take a look into the component, it expects its `redux` prop to be an ImmutableJS data structure. At the moment, we aren't passing anything in as a prop so the `getIn` function is undefined.

We can fix that very easily by `import`ing `fromJS` and passing our `<App />` an empty prop of `redux`:

```
import React from 'react';
import renderer from 'react-test-renderer';
import { fromJS } from 'immutable';
import { App } from '../App';

describe('components', function() {
    describe('<App />', function() {
        it('renders correctly', function() {
            var tree = renderer.create(<App redux={fromJS({})} />).toJSON
();
          expect(tree).toMatchSnapshot();
        });
    });
});
```

Awesome, this totally works! When you now run your tests you'll see a new directory inside the `__tests__` directory called `__snapshots__`. It should contain a single file called `components.test.js` that has an export for our `App` component and some HTML as a string.

```
// __tests__/__snapshots__/components.test.js

exports[`components <App /> renders correctly 1`] = `
<div>
  <h1>
    Weather
  </h1>
  <form
    onSubmit={[Function anonymous]}>
    <label>
      I want to know the weather for
      <input
        onChange={[Function anonymous]}
        placeholder="City, Country"
        type="text"
        value={undefined} />
    </label>
  </form>
</div>
`;
```

Now try changing the text in the App component from "I want to know the

weather for" to "I want to know todays weather for" and run `npm run test` again.
This is the output you should see:

```
PASS  src/__tests__/actions.test.js (0.487s)
PASS  src/__tests__/reducer.test.js (0.58s)
FAIL  src/__tests__/components.test.js (1.042s)
● components › <App /> › it renders correctly
 - expected value to match snapshot 1
   - expected + actual

     <div>
       <h1>
         Weather
       </h1>
       <form
         onSubmit={[Function anonymous]}>
         <label>
   -       I want to know the weather for
   +       I want to know todays weather for
           <input
             onChange={[Function anonymous]}
             placeholder="City, Country"
             type="text"
             value={undefined} />
         </label>
       </form>
     </div>

       at Object.<anonymous> (src/__tests__/components.test.js:10:17)

Snapshot Summary
› 1 snapshot test failed in 1 test file. Inspect your code changes or run
with `npm test -- -u` to update them.

snapshot failure, 1 test failed, 19 tests passed (20 total in 3 test suite
s, run time 1.347s)
```

Awesome, Jest caught the changes in the output of our App component and immediately notified us of a potential error! If we wanted to make this the correct text, all we would have to do is run `npm run -- -u` (`-u` stands for "update snapshots") and Jest would recognize this output as the correct one!

Let's try to do the same thing for our `Plot`. First, import it and add the testing

structure:

```
import Plot from '../Plot.js';

describe('components', function() {
  describe('<App />', function() {/* … */});

  describe('<Plot />', function() {
    it('renders correctly', function() {

    });
  });
});
```

> We don't need to export the Plot separately here since this isn't `connect`ed anyway!

Now try adding a first snapshot:

```
import Plot from '../Plot.js';

describe('components', function() {
  describe('<App />', function() {/* … */});

  describe('<Plot />', function() {
    it('renders correctly', function() {
      const tree = renderer.create(<Plot />).toJSON();
      expect(tree).toMatchSnapshot();
    });
  });
});
```

Run `npm run test` and you'll see this error: `"ReferenceError: Plotly is not defined"`. We use `Plotly.newPlot` in the `drawPlot` method, so at least we know that's being ran!

We need to pretend to Jest that `Plotly` exists for our component. We do this by adding a new field to the `global` variable which the `Plot` component will try to get `Plotly` from:

```
import Plot from '../Plot.js';
```

```
describe('components', function() {

  describe('<App />', function() {/* … */});

  describe('<Plot />', function() {
    global.Plotly = {
      newPlot: () => {}
    };
    it('renders correctly', function() {
      const tree = renderer.create(<Plot />).toJSON();
      expect(tree).toMatchSnapshot();
    });
  });
});
```

Now that that's "defined" (at least we pretend like it is), let's try running `npm run test` again! Another error, this time saying `"TypeError: Cannot read property 'toJS' of undefined"`?

Wait, didn't we have a similar error before? Exactly, this is an ImmutableJS problem again! Our `Plot` expects two immutable data structures to be passed in as `xData` and `yData`. Soo, let's do that? We have `fromJS` already imported from before, so we just add those as props:

```
import Plot from '../Plot.js';

describe('components', function() {
  describe('<App />', function() {/* … */});

  describe('<Plot />', function() {
    global.Plotly = {
      newPlot: () => {}
    };
    it('renders correctly', function() {
      const tree = renderer.create(<Plot xData={fromJS({})} yData={fromJS
({})} />).toJSON();
      expect(tree).toMatchSnapshot();
    });
  });
});
```

Nothing fancy, let's see what happens now! Ugh, *another* error saying `"ReferenceError: document is not defined"`. How many more errors will we get??

get??

As a short aside, this is what happens when you integrate general JS libraries with React. The nice thing is, React is just JavaScript so contrary to some other frameworks it's possible! That doesn't mean it's easy though, but we're almost through it!

Let's get on with it, since the `react-test-renderer` renders the components in a non-browser context (the command line) we need to tell Jest that the `document` variable we use for `document.getElementById('...').on('...')` isn't undefined for us!

We do this again like `Plotly` above by attaching a new property to `global`:

```
import Plot from '../Plot.js';

describe('components', function() {
  describe('<App />', function() {/* … */});

  describe('<Plot />', function() {
    global.Plotly = {
      newPlot: () => {}
    };
    global.document = {
      getElementById: function() { return {
        on: function() {}
      }}
    };
    it('renders correctly', function() {
      const tree = renderer.create(<Plot xData={fromJS({})} yData={fromJS
({})} />).toJSON();
      expect(tree).toMatchSnapshot();
    });
  });
});
```

Now when you run this, what do you see?!

```
 PASS  src/__tests__/actions.test.js (0.528s)
 PASS  src/__tests__/reducer.test.js (0.623s)
 PASS  src/__tests__/components.test.js (0.947s)

Snapshot Summary
› 1 snapshot written in 1 test file
```

```
› 1 snapshot written in 1 test file.

21 tests passed (21 total in 3 test suites, 2 snapshots, run time 1.242s)
```

Yesss!!! 🎉 We have now successfully tested our entire application, whenever something breaks we now immediately know!

Outro

Congratulations, if you've reached this far. You are now ready to go and build beautiful apps using React and Redux.

Congratulations, you've now built your first real-world React application!

We've gone over a lot, starting off with learning the basics of React, integrating a standard JavaScript library with it, fetching and managing data from an API, managing state with Redux and ImmutableJS all the way to testing our entire app!

This is everything you need to know to get started building your own app with React. Go out there and create amazing things!

License

The MIT License (MIT)

Copyright © 2016 Plotly

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.